



Embodied AI

Lecture 3-4: Machine Perception

Junwei Liang

AI Thrust

HKUST (Guangzhou)

Logistics: Part 1

- Course website

- <https://hkust-aiaa4220.github.io/fall2026/lecs.html>
- You can see the **course schedule, homework links, download slides, policy**, etc.
- The slides will be available before each class

You can download the slides now from the schedule website for today's lecture if you want to take note on it

Embodied AI

AIAA 4220, Fall 2026

Instructors: [Junwei Liang](#)



Home Schedule	
Lecture:	Date and Time: 09:00AM - 11:50AM, Monday Location: Rm 233, E1 Websites: Instructure Zoom: 628-334-1826 PW: 12345 (In-person expected) (Recordings will be available after each class)
Course Description:	This course aims to provide students with key principles and algorithms to build modern Embodied AI systems. Key topics include machine perception, embodied navigation and manipulation algorithms. Through this course, students will learn and practice the foundational principles, techniques, and tools to build new Embodied AI systems.
Target Audience/ Prerequisites:	This is a course primarily for junior and senior students.
Course Work:	Grading Grading will be based on best 5 of 6 in-class quizzes (25%), one paper presentation (25%), one machine perception homework (25%) and one simulation project (25%). • The in-class quizzes are designed to complement the lectures and making sure you understand the key knowledge about the course. • There are 6 in-class quizzes over the semester. We will take your best 5 out of the 6 quizzes as your grade. In-Class Quizzes

For new students, check out slides from last lectures to see the details.

Recap - What is Embodied AI?

- Embodied AI is the intersection of **machine learning and robotics**
- The study of **training optimal solutions using sensory data** for robots to act and achieve goals
- **Perception** and decision making are the key



How to Start Surveying a Research Direction?

1. General Strategy

• Dataset-Centric Approach

Identify primary benchmark datasets driving the specific field .

• Literature Review

Check highly-cited and recent state-of-the-art papers using these datasets .

• Establish Baselines

Map out standard baseline models, metrics, and experimental protocols .

2. Primary Conferences & Venues

Computer Vision (Inc. Embodied AI)

CVPR, ICCV, ECCV, SIGGRAPH (3D Vision, NeRF/GS)

Machine Learning

NeurIPS, ICML, ICLR, AAAI, IJCAI

(Learning-based) Robotics

ICRA, IROS, **RSS**, **CoRL** / RA-L, T-RO, Science Robotics

Key Workshops

CVPR AI City Challenge, Autonomous Driving, Ego4D; NeurIPS ML for Autonomous Driving

★ De Facto Top Embodied AI Conferences: **NeurIPS**, **RSS**, **CoRL**

3. Resources

Survey Papers

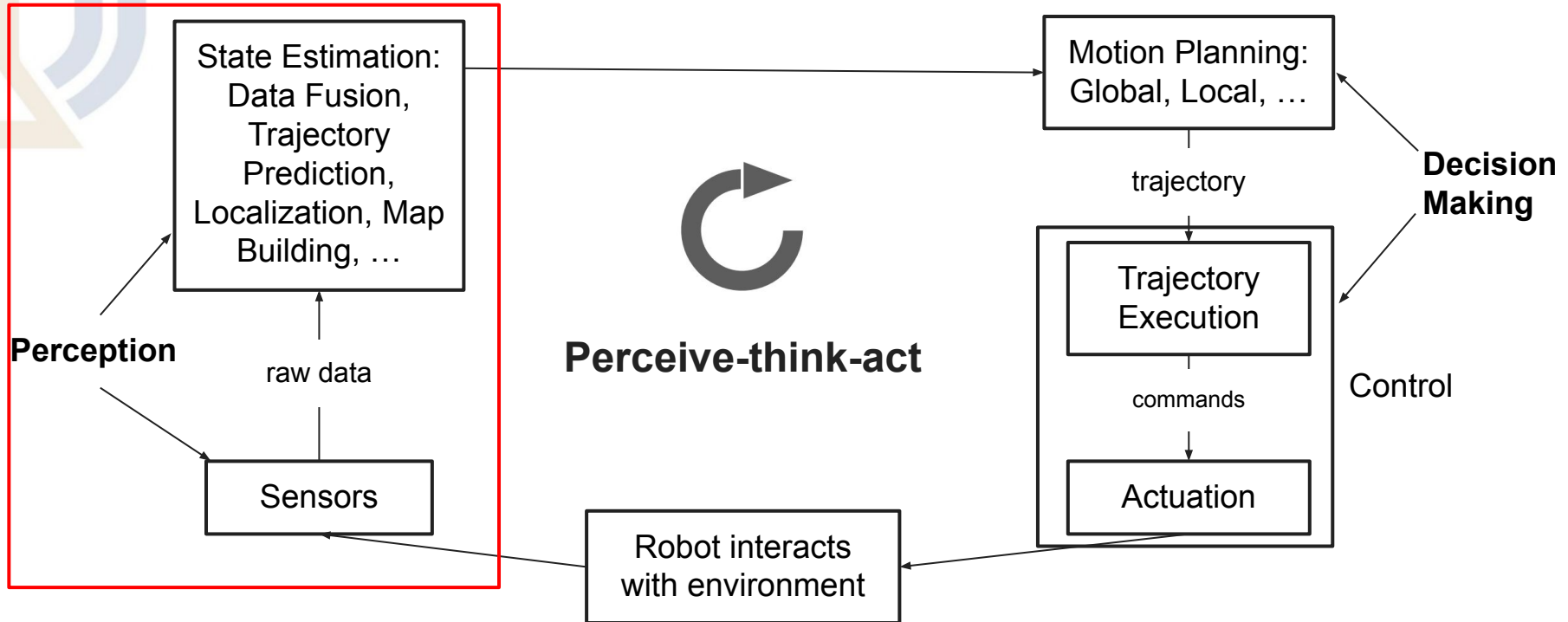
Awesome Lists

Papers With Code

Codex / Repos

Recap - The Embodied AI Paradigm

- The Perceive-think-act circle



Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Key models
 - CNN
 - Dense prediction
 - Attention model
 - Large-scale vision-language learning
 - Geometric computer vision
 - Bird-Eye-View perception
 - Ego-View perception

Computer Vision Background - Image and Video Storage

- Digital image
 - A set of matrices of numbers
- Grayscale: a single matrix of numbers
 - Each number represents the intensity of the image at a specific location in the image
 - Interpreted as the shade of grey at that location
 - Grayscale images are typically derived from color images



What we see

0	3	2	5	4	7	6	9	8
3	0	1	2	3	4	5	6	7
2	1	0	3	2	5	4	7	6
5	2	3	0	1	2	3	4	5
4	3	2	1	0	3	2	5	4
7	4	5	2	3	0	1	2	3
6	5	4	3	2	1	0	3	2
9	6	7	4	5	2	3	0	1
8	7	6	5	4	3	2	1	0

What a computer sees

Source: S. Narasimhan

Computer Vision Background - Image and Video Storage

- Digital image - Grayscale

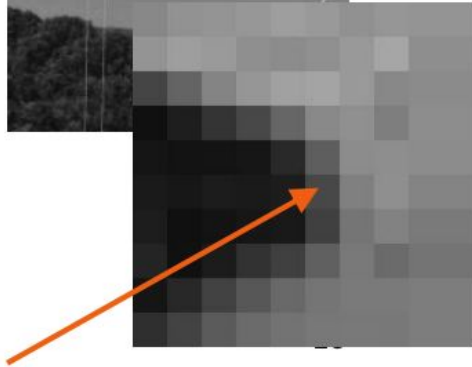
What we see



Only a single number need
be stored per pixel

What the computer "sees"

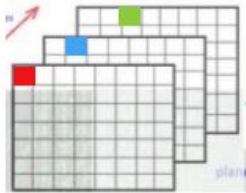
147	157	158	161	168	164	156	159	158	158
157	166	163	158	154	155	162	170	154	154
78	111	143	162	171	173	163	149	150	150
13	36	60	82	116	145	155	136	151	151
21	18	19	22	46	106	151	152	151	151
27	25	29	28	34	87	141	151	143	143
31	11	19	22	26	72	128	141	133	133
40	15	29	53	72	108	131	124	128	128
22	46	75	99	116	128	133	133	136	136
55	79	103	122	125	130	132	132	136	136



Picture Element (PIXEL)
Position & gray value (scalar)

Computer Vision Background - Image and Video Storage

- Digital image - Color images



Each pixel represented by a triad of numbers

Picture Element (PIXEL)
Position & color value (red, green, blue)

Computer Vision Background - Image and Video Storage

- Digital image - Color images - RGB colorspace



Example from wikipedia



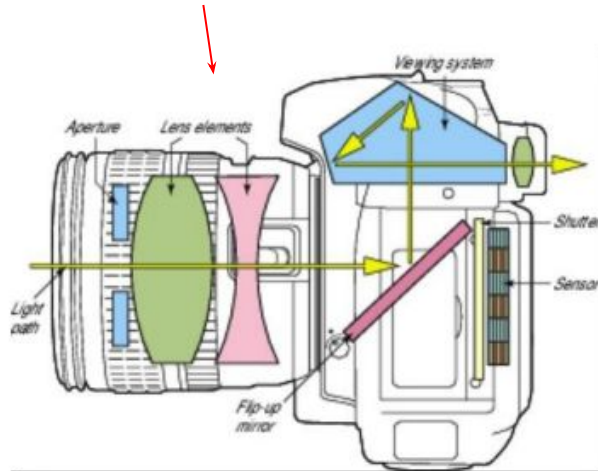
RGB colorspace:

The three matrices represent R, G and B

Computer Vision Background - Image and Video Storage

- What happens after camera capture?
 - The digital image is not stored in its raw form
 - Image is processed in multiple stages before it is stored

Pinhole camera model with lens:
we cover this in last lecture

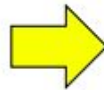


Computer Vision Background - Image and Video Storage

- What happens after camera capture?
 - The digital image is not stored in its raw form
 - Image is processed in multiple stages before it is stored
 - White balancing
 - Demosaicing
 - Interpolation for colors
 - Gamma correction
 - And other processing steps

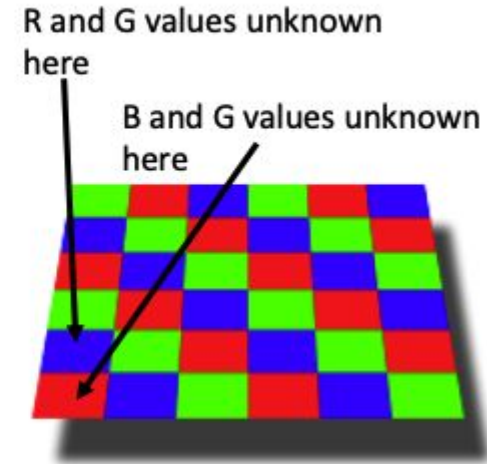
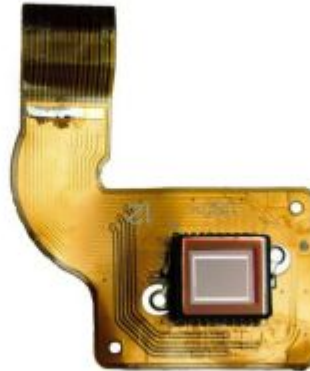
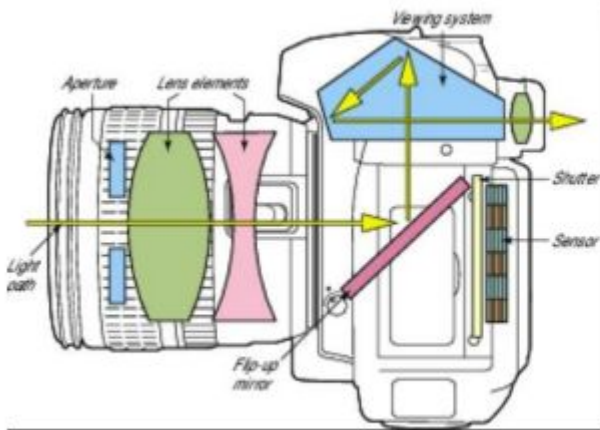
Computer Vision Background - Image and Video Storage

- White balancing
 - Camera automatically adjusts individual colors to give you a more “neutral” image
 - Closer to an image taken under white light



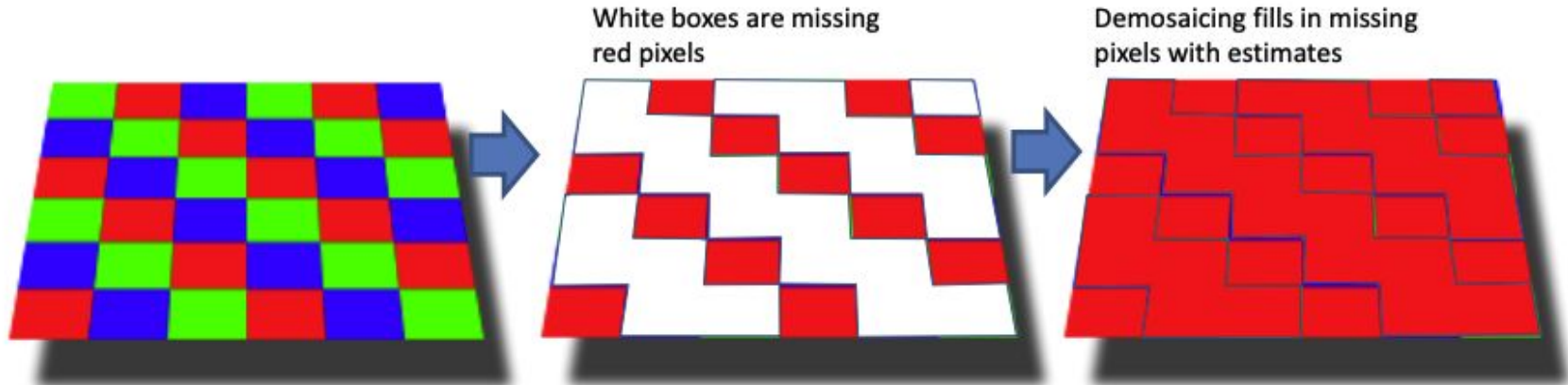
Computer Vision Background - Image and Video Storage

- Demosaicing
 - An NxM digital camera does not actually capture NxM pixels for each of the three colors
 - R, G and B sensors alternate
 - In any pixel, the value of only one of the colors is known, the other two are unknown



Computer Vision Background - Image and Video Storage

- Demosaicing
 - For each color, the missing pixels must be filled in
 - Done by estimating them from their neighbors
 - A linear combination of the nearest present pixels
 - This process is called demosaicing
 - Actually similar to the super-resolution task!



Computer Vision Background - Image and Video Storage

- Gamma correction
 - Pixel values (of all colors) are compressed through a non-linearity
 - This reduces the dynamic range of pixel values
 - Can be stored using fewer bits



Computer Vision Background - Image and Video Storage

- Storage: The Exchangeable Image Format (EXIF)
 - A standardized format for images
 - Agreed upon by a number of standardization bodies
 - Specifies formats for images, sounds, and other tags used by digital image recorders (cameras, scanners, etc.)
 - Standard includes format specifications for
 - JPEG images
 - TIFF images (Tagged Image File Format)
 - RIFF wav for audio files (Resource Interchange File Format)
 - Appended metadata

Computer Vision Background - Image and Video Storage

- Storage: The Exchangeable Image Format (EXIF)
 - Example
 - Note the camera focal length! (and other intrinsic parameters)

Tag	Value
Manufacturer	CASIO
Model	QV-4000
Orientation (rotation)	top-left [8 possible values ^[20]]
Software	Ver1.01
Date and time	2003:08:11 16:45:32
YCbCr positioning	centered
Compression	JPEG compression
X resolution	72.00
Y resolution	72.00
Resolution unit	Inch
Exposure time	1/659 s
F-number	f/4.0
Exposure program	Normal program
Exif version	Exif version 2.1
Date and time (original)	2003:08:11 16:45:32

from wikipedia

Date and time (digitized)	2003:08:11 16:45:32
Components configuration	Y Cb Cr –
Compressed bits per pixel	4.01
Exposure bias	0.0
Max. aperture value	2.00
Metering mode	Pattern
Flash	Flash did not fire
Focal length	20.1 mm
MakerNote	432 bytes unknown data
FlashPix version	FlashPix version 1.0
Color space	sRGB
Pixel X dimension	2240
Pixel Y dimension	1680
File source	DSC
Interoperability index	R98
Interoperability version	(null)

Computer Vision Background - Image and Video Storage

- What happens after camera capture?
 - The digital image is not stored in its raw form
 - Image is processed in multiple stages before it is stored
 - White balancing
 - Demosaicing
 - Interpolation for colors
 - Gamma correction
 - And other processing steps
 - **Image compression**, JPEG, PNG...

Computer Vision Background - Image and Video Storage

- Video cameras
 - Just like still image cameras
 - Take multiple shots automatically at a fixed rate per second
 - The rate is decided based on persistence of vision
 - Usually 30 photos/second (frame-per-second, FPS)
 - Sound is recorded alongside and time synchronized with the photo frames



- Global shutter, rolling shutter
 - Shutter speed 快门/曝光时间
- 30 FPS: shutter speed at most $1/30 = 33.3 \text{ ms}$
- When we do tennis ball tracking, shutter speed of $< 1 \text{ ms}$ is needed.

Computer Vision Background - Image and Video Storage

- Video cameras
 - Codec
 - scheme to **compress** images for storage and distribution and to **decompress** the compressed images for display
 - Why?
 - Raw HD video can take up to 400GB per hour..
 - Image: JPEG, PNG,...
 - Video: H.264, MPEG-4, ...
 - Can be represented in 2 spatial and 1 temporal dimension
 - Coding exploits **redundancies** in spatial and temporal domains

Computer Vision Background - Image and Video Storage

- Video cameras
 - Codec - Video coding
 - Spatial coding
 - Also called intra-coding
 - Exploits redundancies within a frame (JPEG, etc.)
 - Temporal coding
 - Also called predictive coding
 - Exploits redundancies across frames

Computer Vision Background - Image and Video Storage

- Video cameras
 - Codec - Video coding
 - 3 types of frames
 - **I-frame** (Intra-coded), P-frame (Predicted) and B-frame (Bi-directionally predicted)
 - I-frames are coded using a JPEG like scheme. Exploits spatial redundancies (as in image coding). The first frame of a video is an I-frame
 - P-frames, B-frames: Only changes from the reference are stored.
 - I-frames are inserted at regular intervals or inserted based on the motion in the video
 - These compressions are not lossless (hence there are artifacts). Sometimes our recognition model may be affected

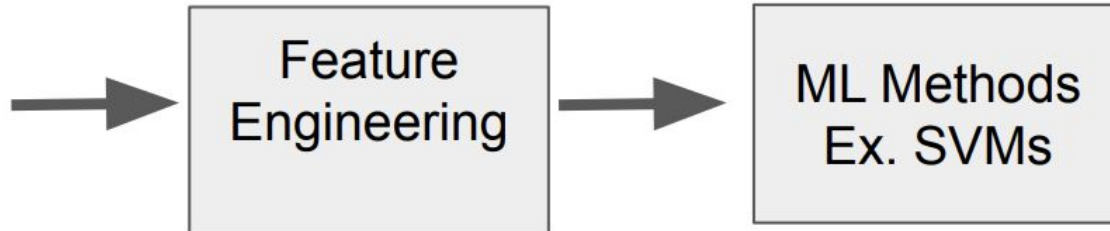


Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Key models
 - CNN
 - Dense prediction
 - Attention model
 - Large-scale vision-language learning
 - Geometric computer vision
 - Bird-Eye-View perception
 - Ego-View perception

Computer Vision Background - Features

- Before convolutional neural network (CNN) model
 - Feature engineering based on heuristics
 - Local descriptors
 - Global representation
 - Then apply ML methods like support vector machines
 - Covered in AIAA 5032 (Foundations of AI) / AIAA 2205 (Intro to AI)



Computer Vision Background - Features

- Local descriptors
 - histogram of oriented gradients (HOG), Scale-Invariant Feature Transform (SIFT), etc.

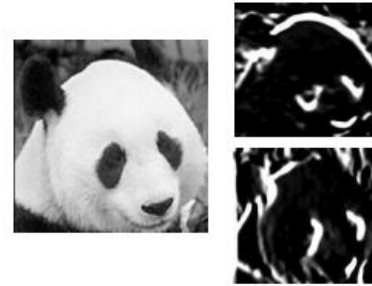
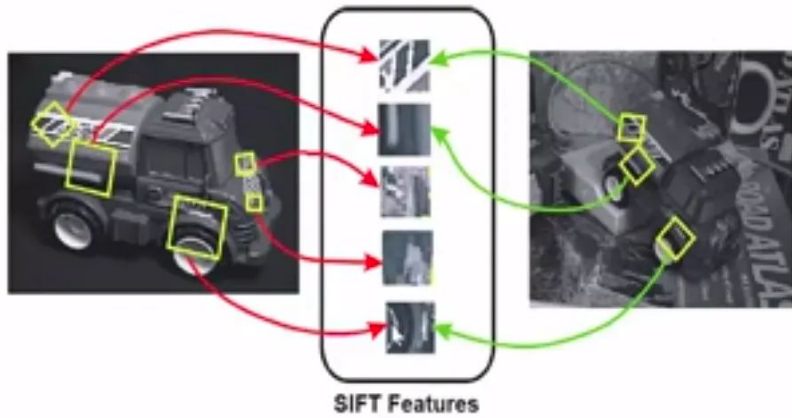
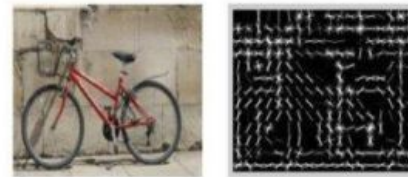


Image gradient



Edge detection



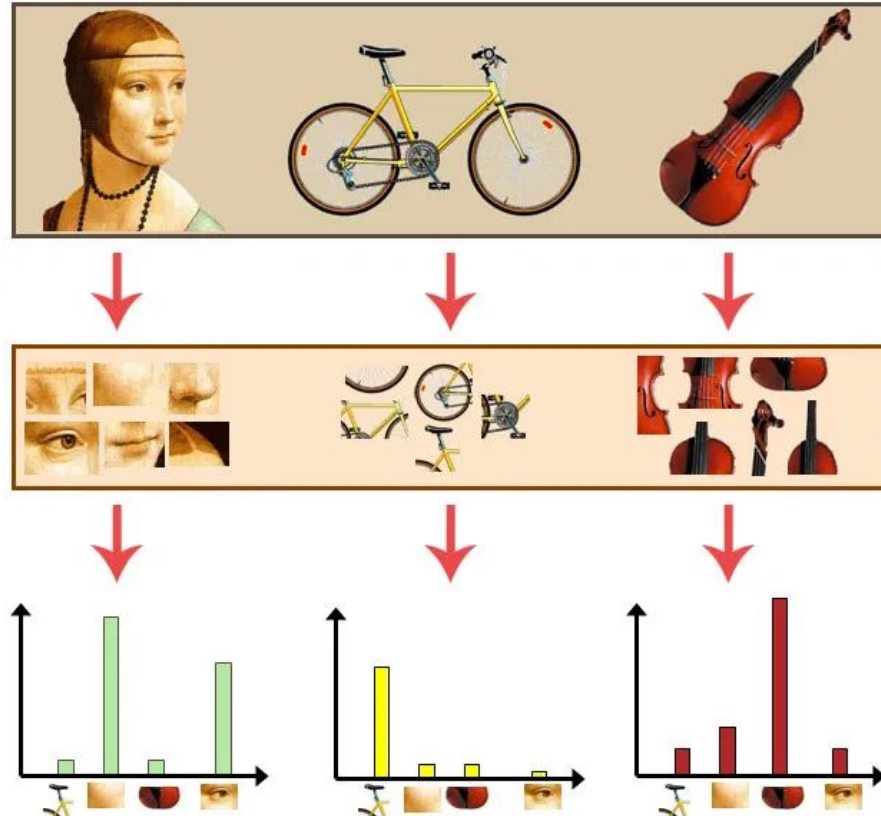
Histograms of
Oriented Gradients



Optical Flow

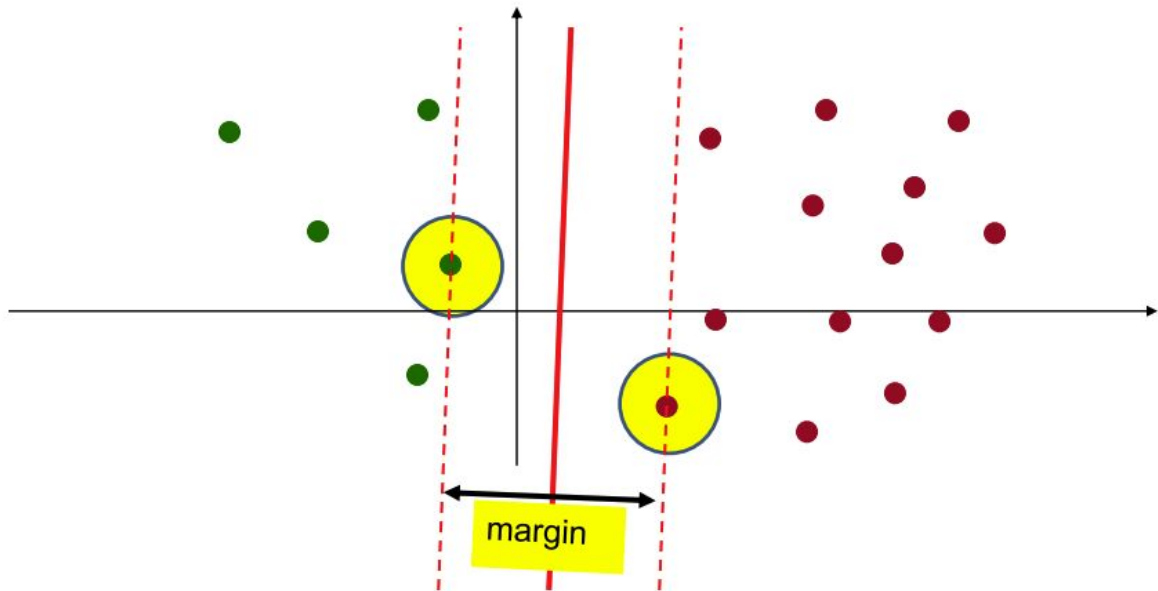
Computer Vision Background - Features

- Global representation
 - Bag-of-visual-words
 - Local descriptors
 - Clustering
 - K-means
 - Got fixed length vector representation for any size image/video



Computer Vision Background - Features

- Global representation
 - Bag-of-visual-words
 - Local descriptors
 - Clustering
 - K-means
 - Got fixed length vector representation for any size image/video
- Then apply ML methods like support vector machines (SVM)

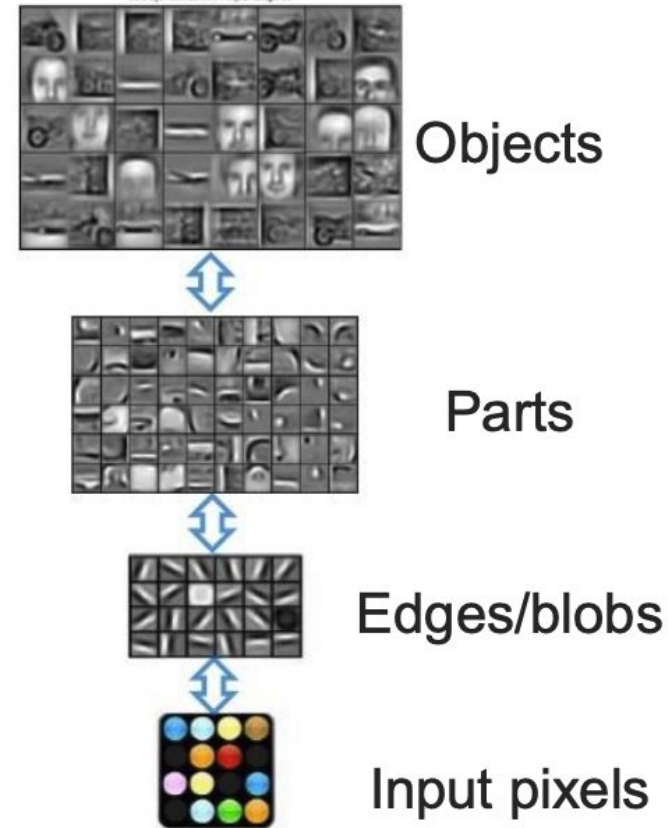


Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Key models
 - CNN
 - Dense prediction
 - Attention model
 - Large-scale vision-language learning
 - Geometric computer vision
 - Bird-Eye-View perception
 - Ego-View perception

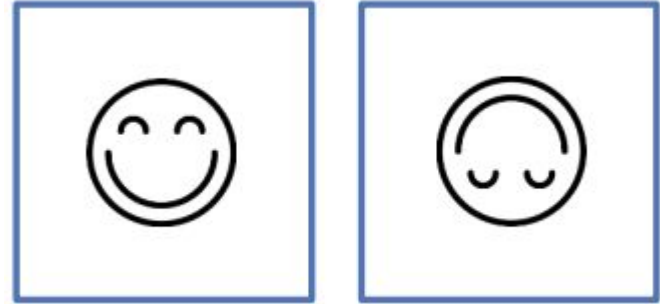
Convolutional Neural Networks

- Design goal
 - building more abstract, hierarchical visual representations
- Key advantages:
 - Inspired from visual cortex (human eye)
 - Encourages visual abstraction
 - Exploits translation invariance
 - Kernels/templates are learned
 - Fewer parameters than MLP



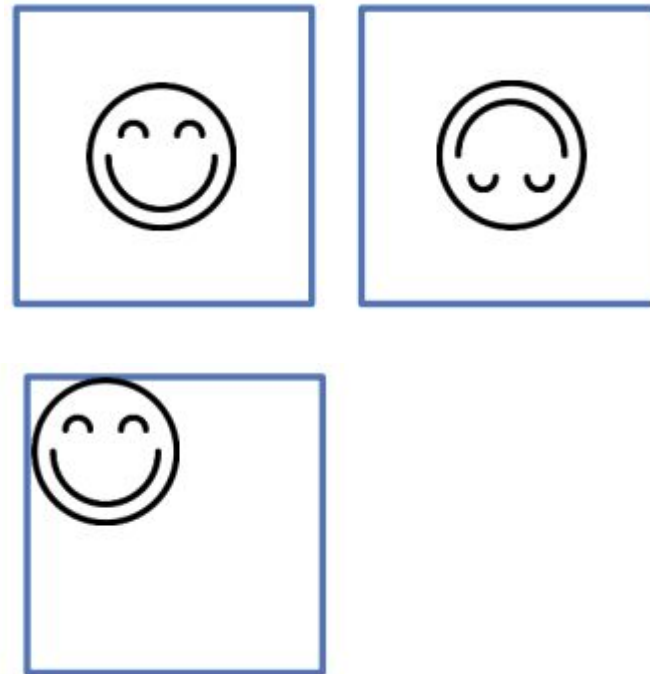
Convolutional Neural Networks

- Translation Invariance
 - 2 data points - which one is up?
 - MLP can easily learn this task



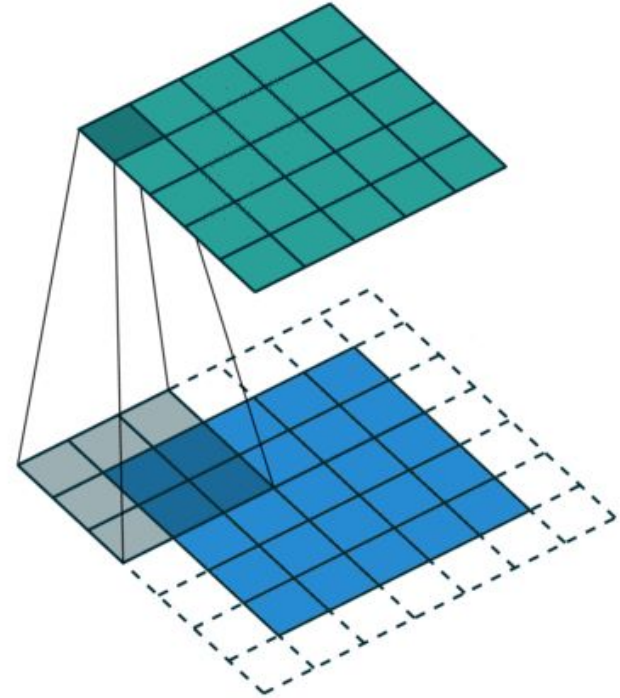
Convolutional Neural Networks

- Translation Invariance
 - 2 data points - which one is up?
 - MLP can easily learn this task
 - What if the face is slightly translated?
 - MLP may fail to recognize this
 - Not translation invariant
 - CNN has translation invariance properties
 - Kernel-based, sliding-window computation



Convolution In Practise

- 1-dimensional CNN:
 - We have a signal of length N
 - Kernel of length K
 - Output will be length $N - K + 1$
- Example
 - Input size (3), kernel/filter size (2), output size (2)
 - $f = [1, 2, 1]$, $g = [1, -1]$, $f * g = [1-2, 2-1] = [-1, 1]$



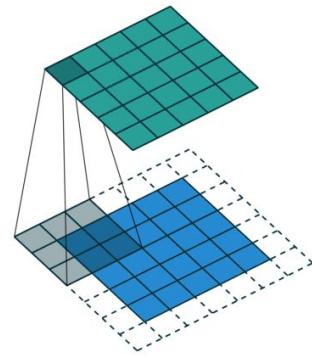
Convolution In Practise

- If we want output to be different sizes, we can **add padding** to the signal:
 - Just add 0s at the beginning and end
 - Example
 - $f = [1, 2, 1], f' = [0, 1, 2, 1], g = [1, -1], f * g = [-1, 1-2, 2-1,] = [-1, -1, 1]$

Convolution In Practise

- If we want output to be different sizes, we can add padding to the signal:
 - Or
 - We can perform **strided (aka, dilated) convolution**: the filter jumps over pixels or samples
 - We have a signal of length N
 - Kernel of length K
 - Output will be length $(N - K)/\text{stride} + 1$
 - Example with stride 2
 - Input of size (6), kernel (2), output (3)
 - $f = [0, 0, 1, 2, 1, 0]$, $g = [1, -1]$, $f * g = [0, -1, 1]$

Convolution In 2D - Example



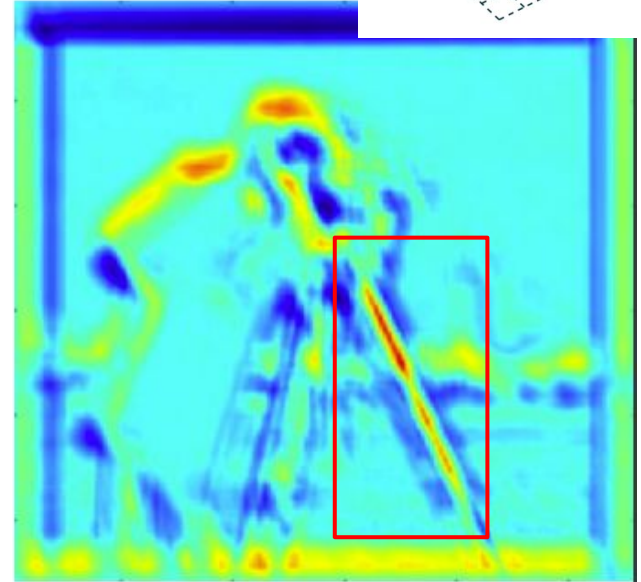
Input image

*



Convolution
kernel

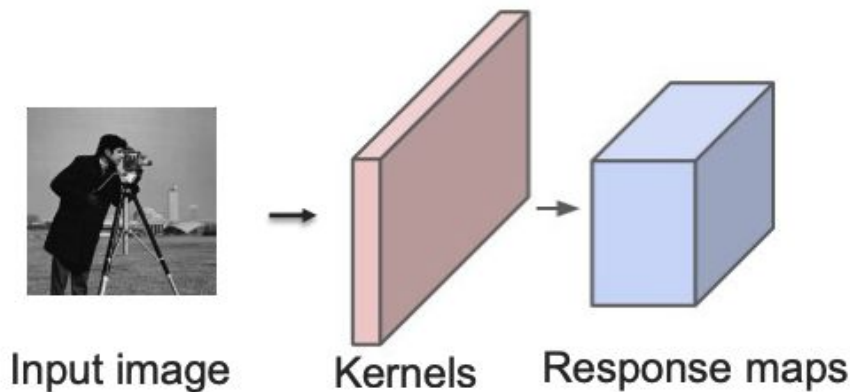
=



Response map

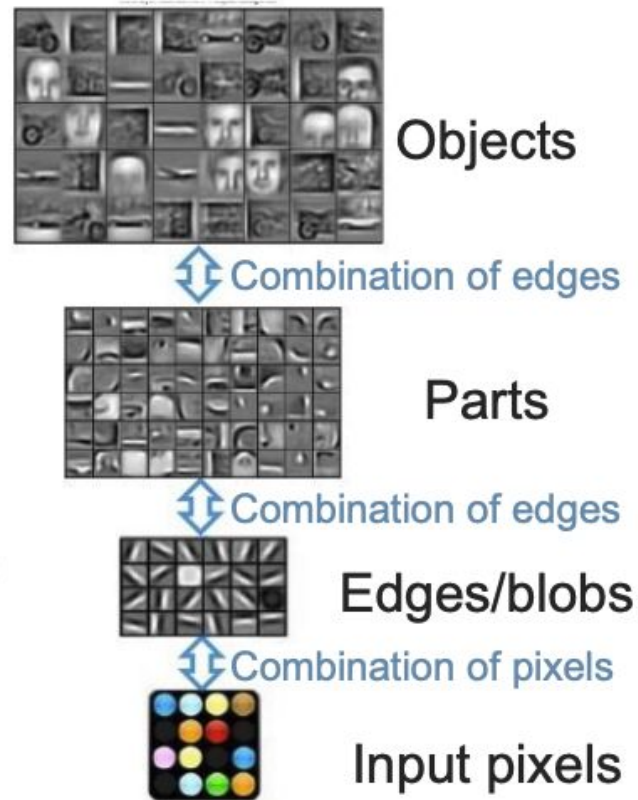
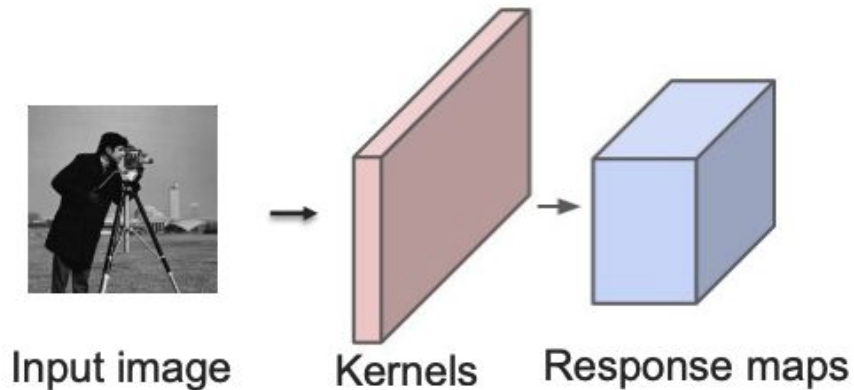
Convolutional Neural Layer

- Can expand this to 2D (or even 3D!)
 - Just need to make sure to link the right pixel with the right weight
- Can expand to multi-channel 2D
 - e.g., for RGB images
- Can expand to multiple kernels/filters
 - Output is not a single image anymore, but a tensor (a 3D matrix)



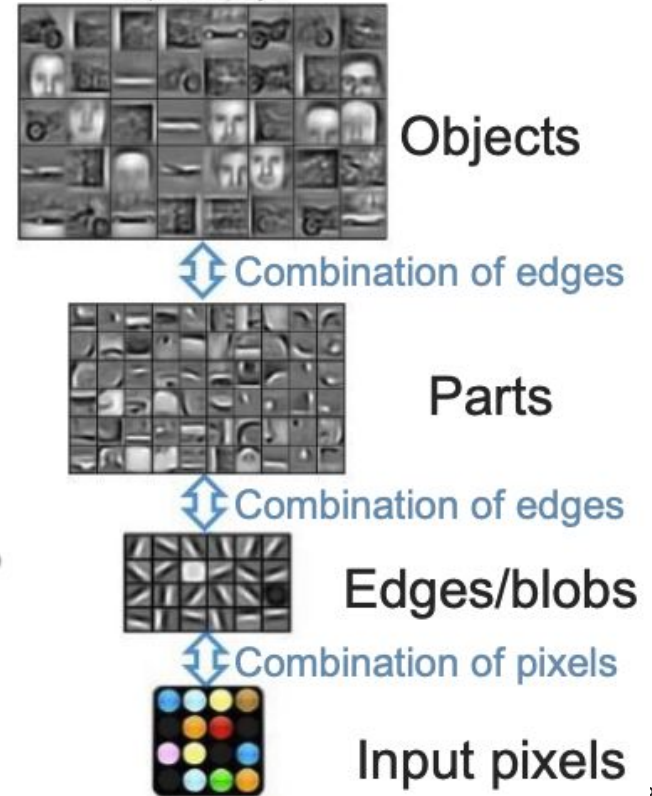
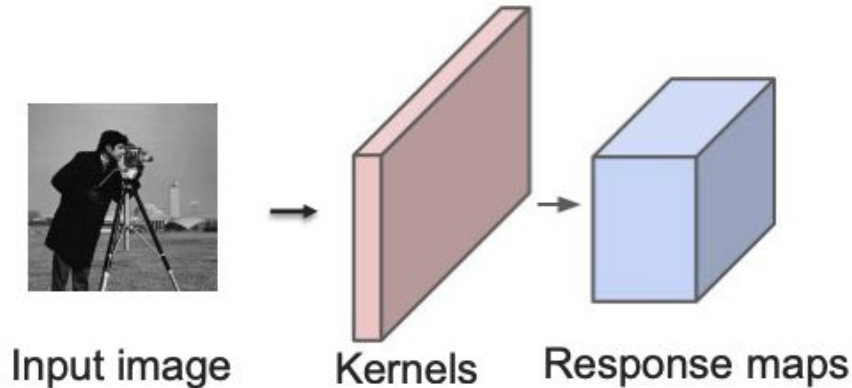
Convolutional Neural Network

- Multiple convolutional layers
 - Allows the network to learn combinations of sub-parts, to increase complexity
-



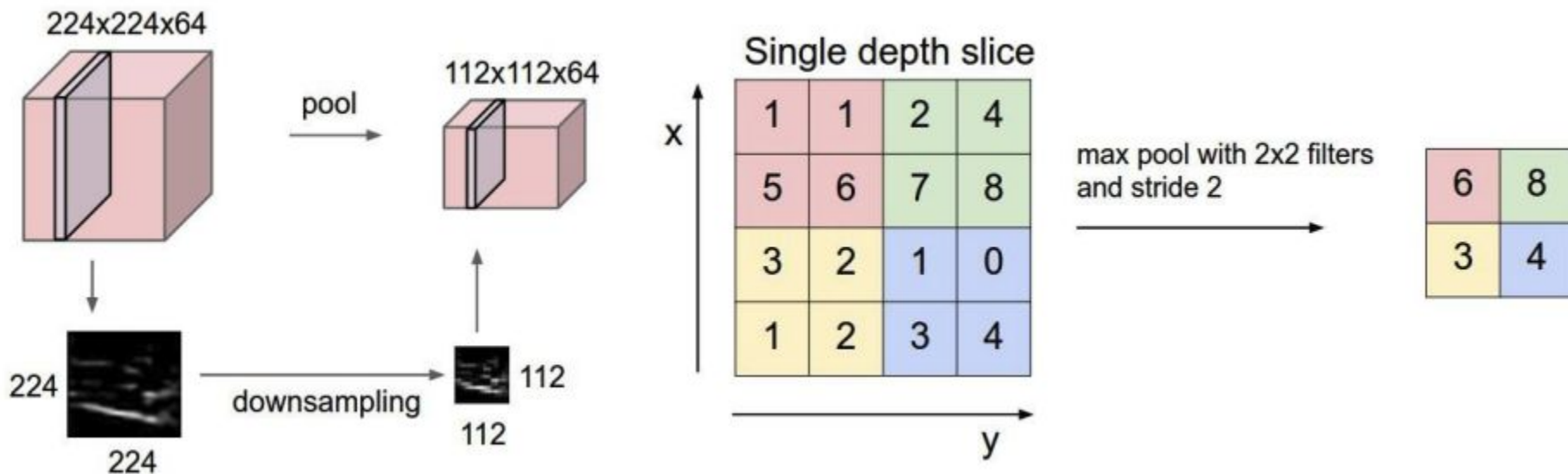
Convolutional Neural Network

- Multiple convolutional layers
 - Allows the network to learn combinations of sub-parts, to increase complexity
- But how to encourage abstraction and summarization?
 - Pooling layer!

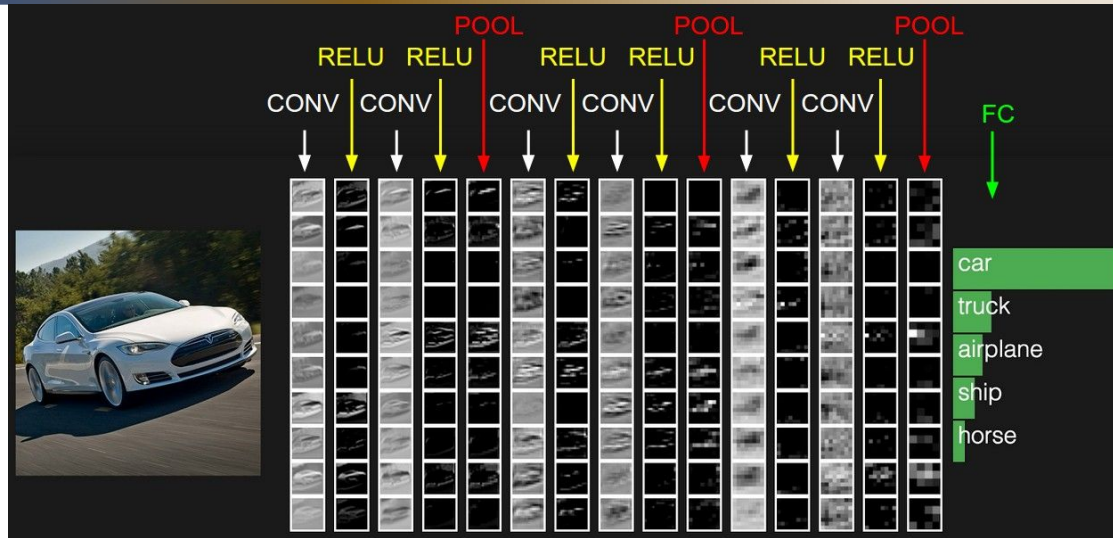


Pooling Layer

- Response map subsampling:
 - Allows summarization of the responses

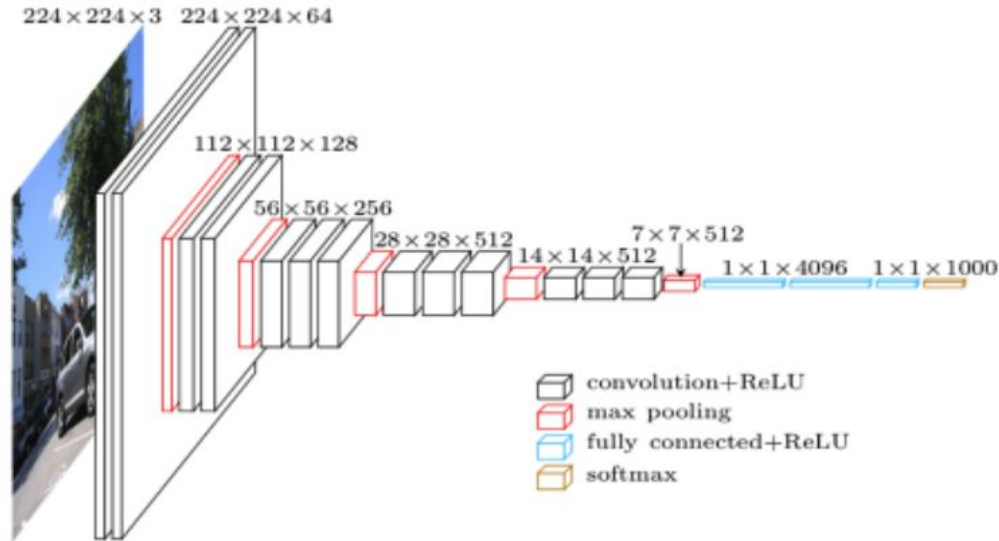


Common CNN Architectures



- Start with a convolutional layer follow by non-linear activation and pooling
- Repeat this several times
- Ends with a fully connected (MLP) layer

Common CNN Architectures

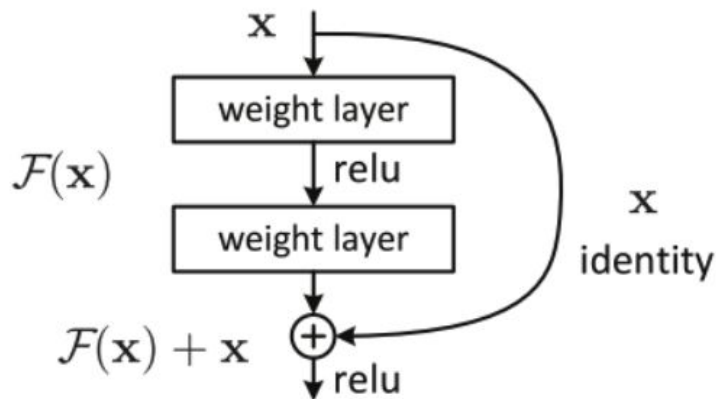


- VGGNet model

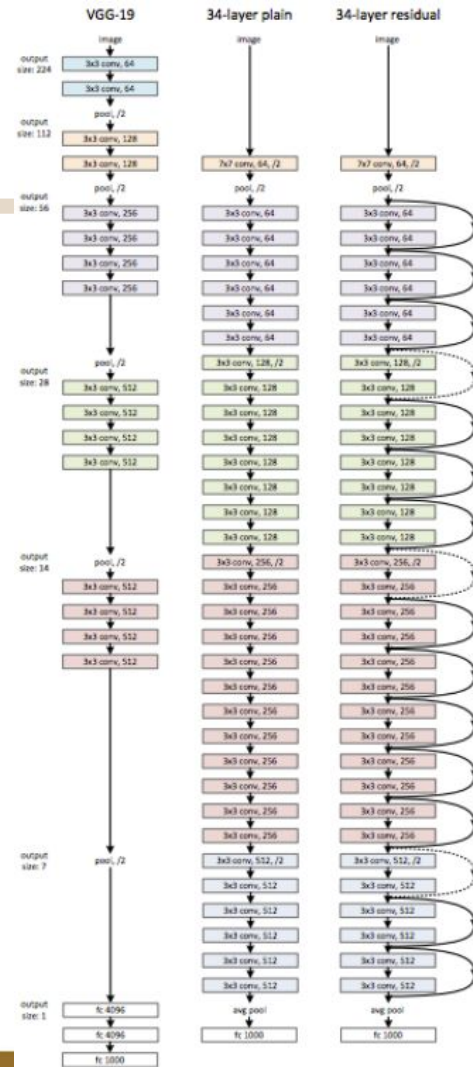
- Used for 1000-way image classification task
- 138 million parameters / 16-19 layers
- *Still relevant today due to its simple connection and fast inference

Residual Networks

- Adding residual connections



- ResNet (He et al., 2015)
 - Up to 152 layers
 - But only 60 million parameters (138M for VGG, 3% better)



Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Key models
 - CNN
 - Dense prediction
 - Attention model
 - Large-scale vision-language learning
 - Geometric computer vision
 - Bird-Eye-View perception
 - Ego-View perception

Dense Prediction

- Image classification vs semantic segmentation

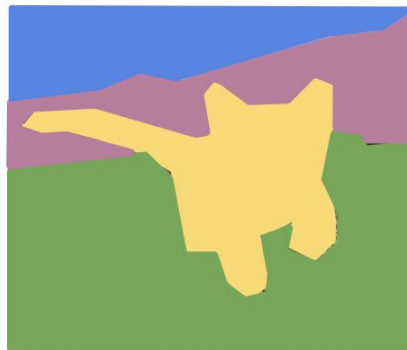
Classification



CAT

No spatial extent

**Semantic
Segmentation**

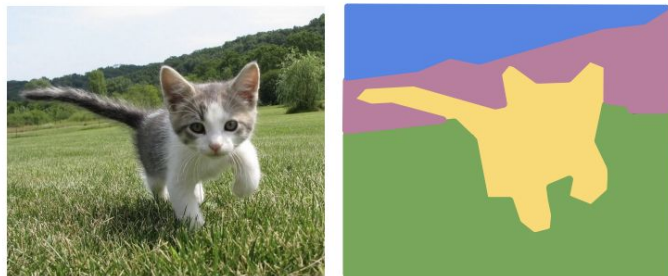


**GRASS, CAT,
TREE, SKY**

No objects, just pixels

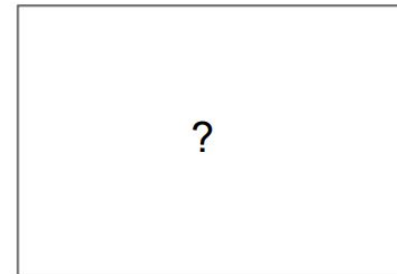
Dense Prediction - Semantic Segmentation

- Problem description



GRASS, CAT,
TREE, SKY, ...

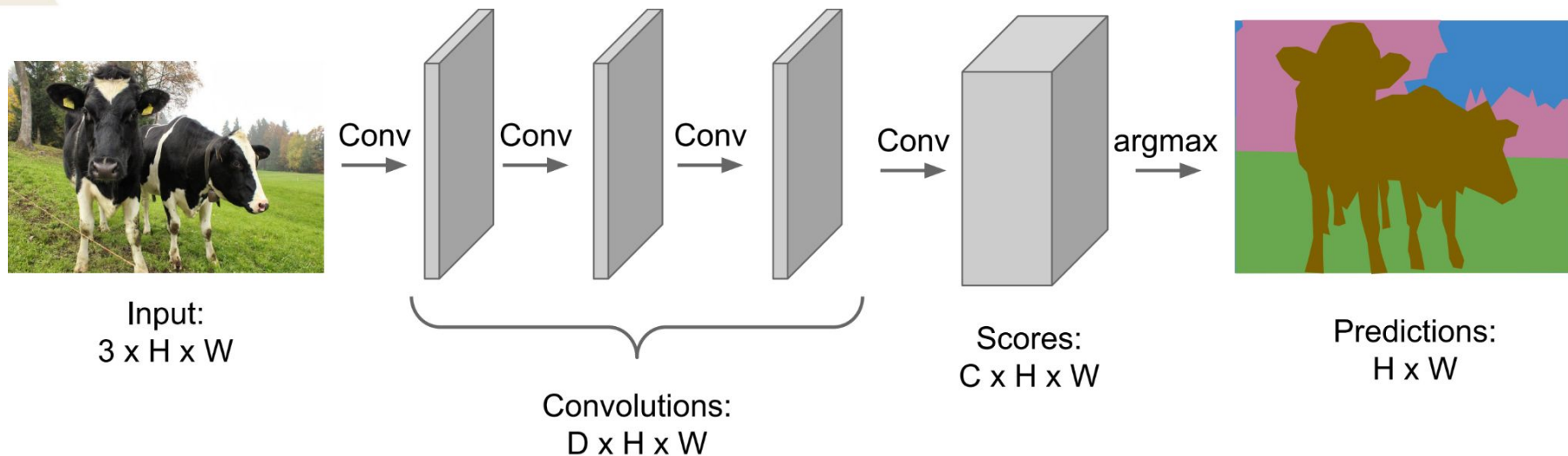
Paired training data: for each training image, each pixel is labeled with a semantic category.



At test time, classify each pixel of a new image

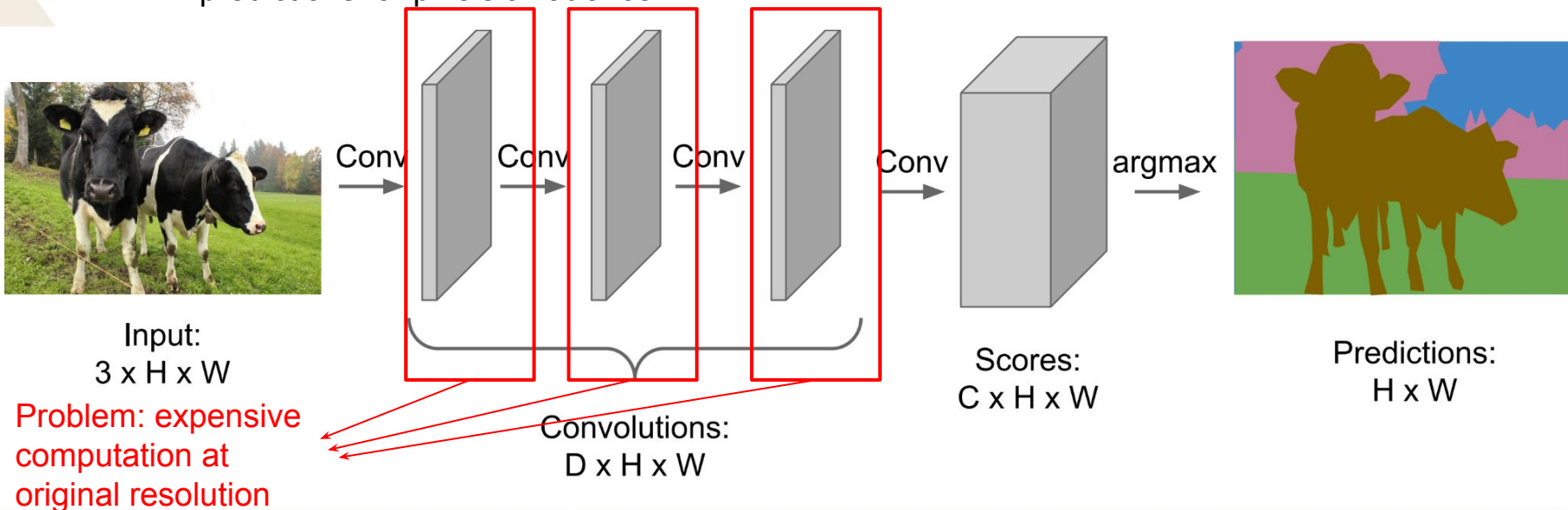
Dense Prediction - Semantic Segmentation

- Idea: Convolution
 - We see that convolution can get same-sized outputs
 - So design a network with only convolutional layers without downsampling poolings to make predictions for pixels all at once!



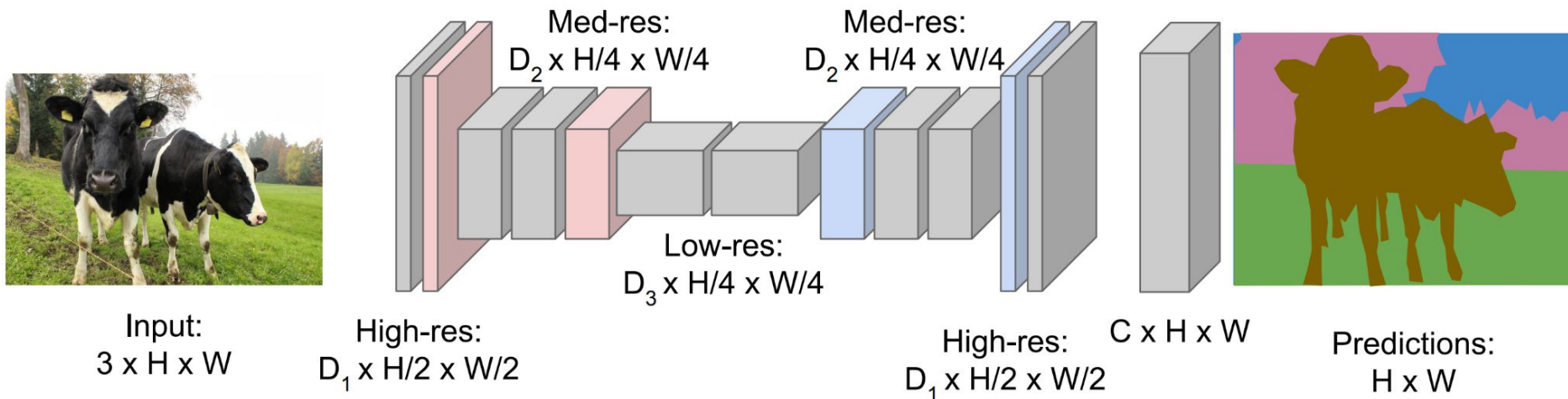
Dense Prediction - Semantic Segmentation

- Idea: Convolution
 - We see that convolution can get same-sized outputs
 - So design a network with only convolutional layers without downsampling poolings to make predictions for pixels all at once!



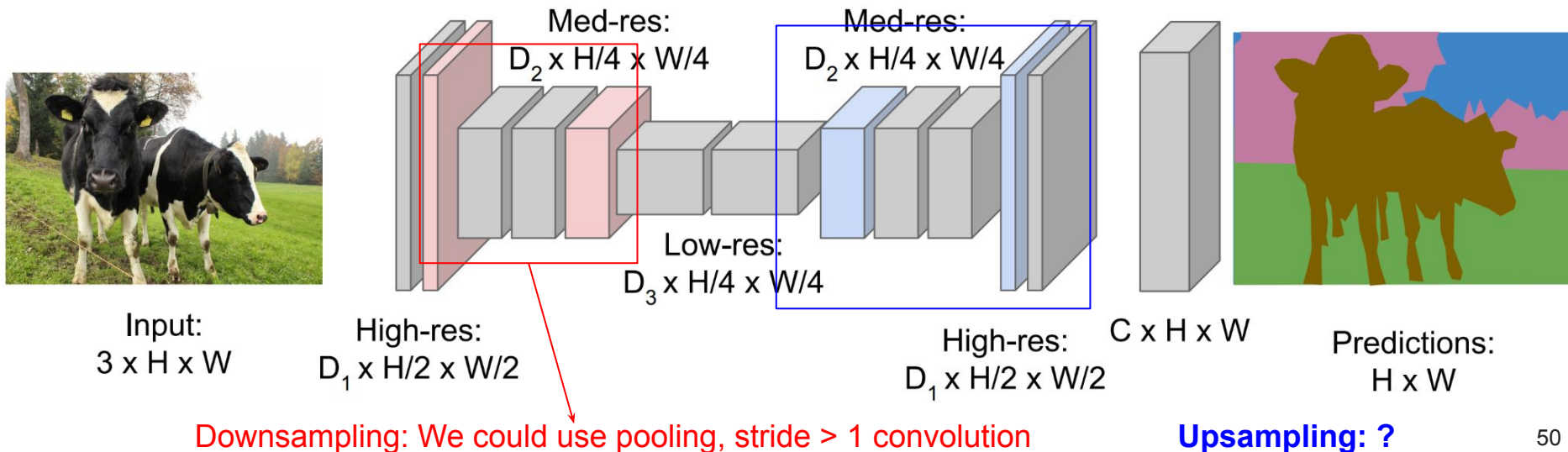
Dense Prediction - Semantic Segmentation

- Idea: convolutional network
 - We see that convolution can get same-sized outputs
 - Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!



Dense Prediction - Semantic Segmentation

- Idea: convolutional network
 - We see that convolution can get same-sized outputs
 - Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!



Dense Prediction - Semantic Segmentation

- Upsampling: Unpooling operation

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

“Bed of Nails”

1	2
3	4



1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input: 2 x 2

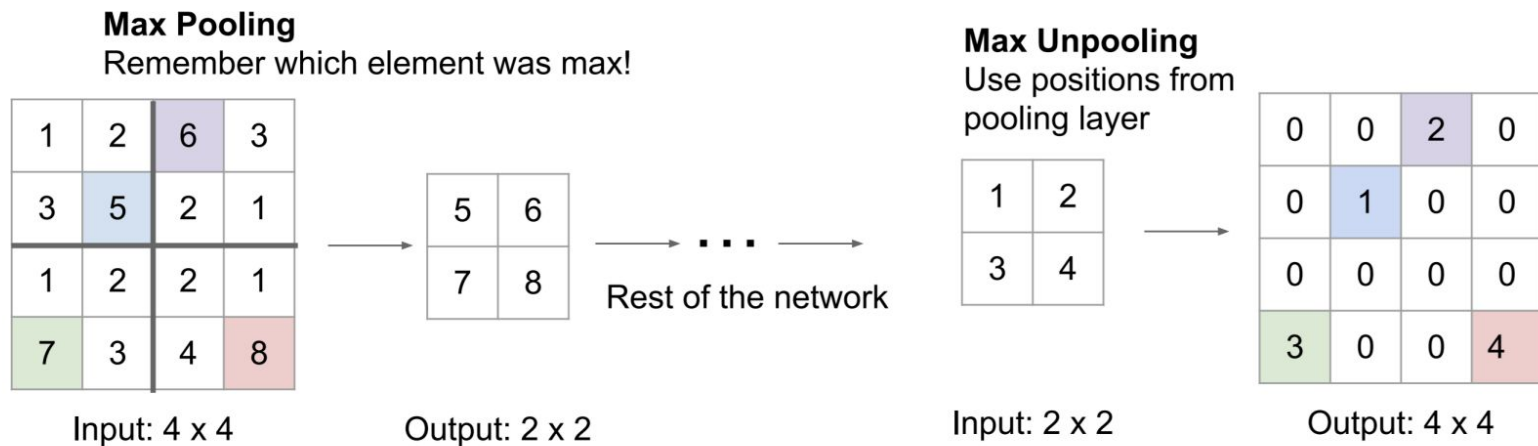
Output: 4 x 4

Put value at a fixed position and fill the rest with zeros

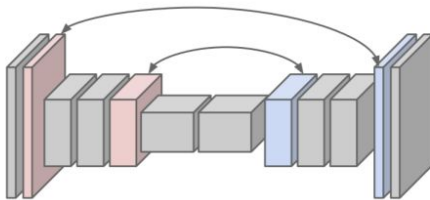
Dense Prediction - Semantic Segmentation

- Upsampling: Max Unpooling

Because segmentation models are often symmetric

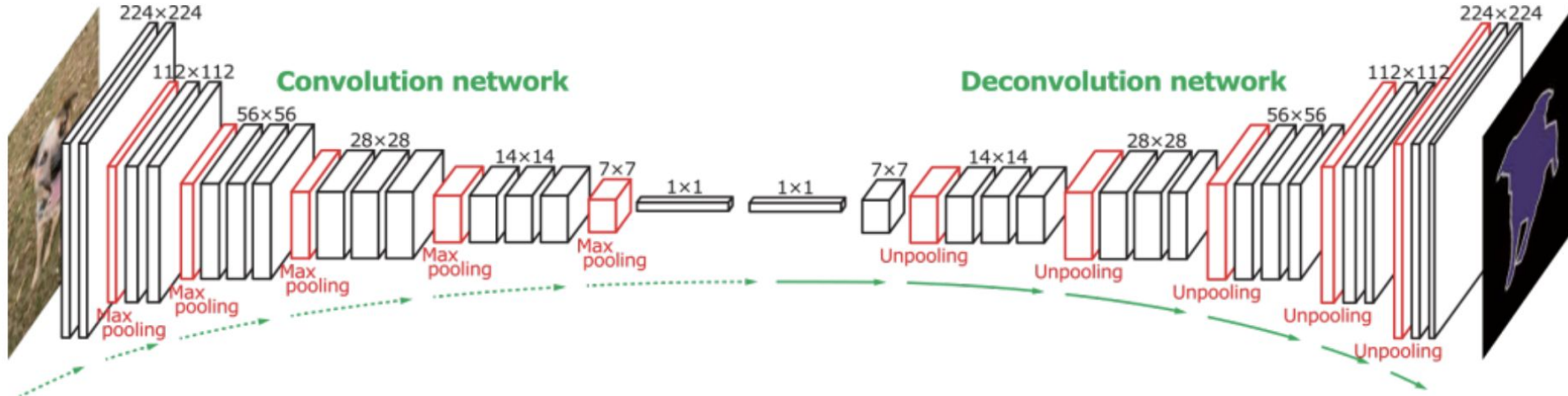


Corresponding pairs of
downsampling and
upsampling layers



Dense Prediction - Semantic Segmentation

- Upsampling: Max Unpooling



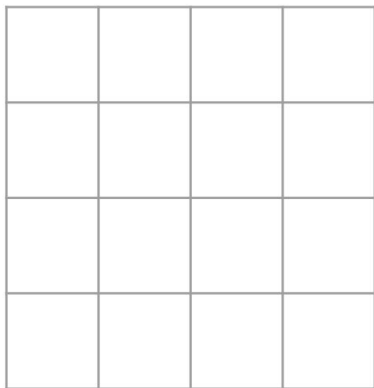
*Learning deconvolution network for semantic segmentation. ICV 2015

Dense Prediction - Semantic Segmentation

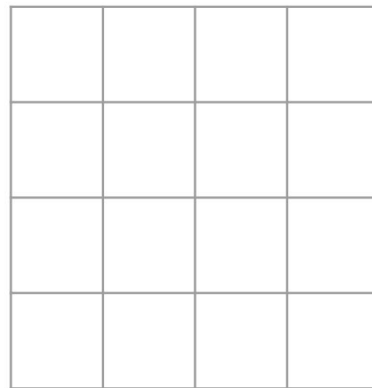
- Upsampling: Transposed Convolution
 - Recall - normal convolution
 - Kernel (3 x 3), stride=1, pad=1 (input 4x4 becomes 6x6, so output size $6-3+1=4 \times 4$)

0	1	1
0	0	0
-1	1	1

Kernel



Input: 4 x 4

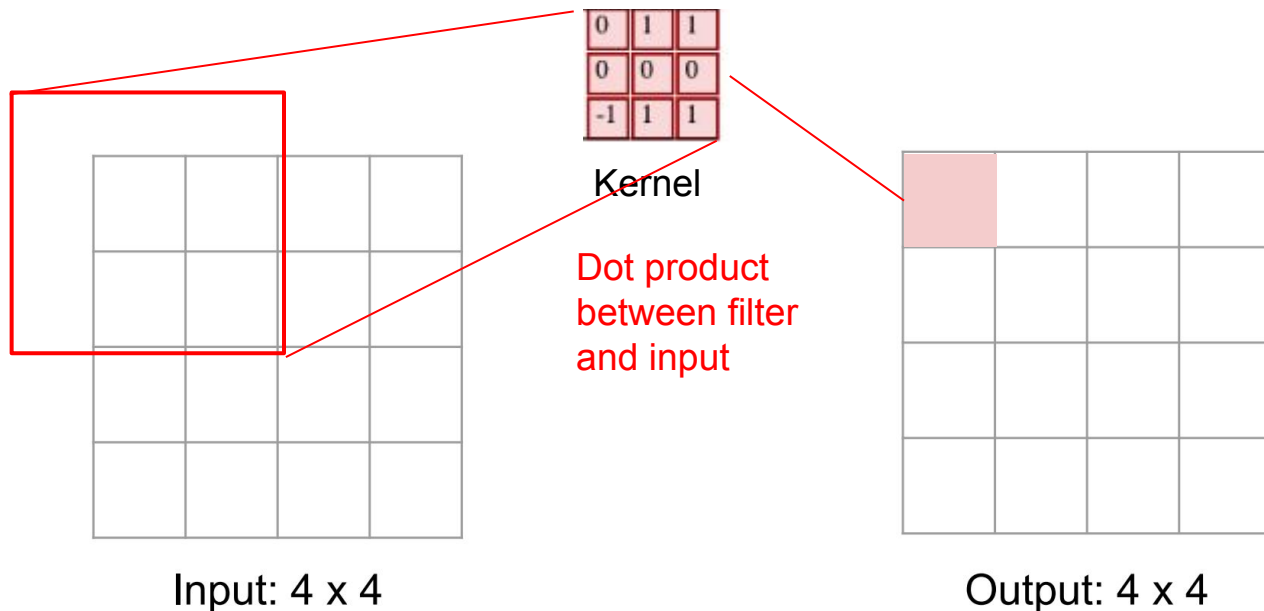


Output: 4 x 4

Dense Prediction - Semantic Segmentation

- Upsampling: Transposed Convolution

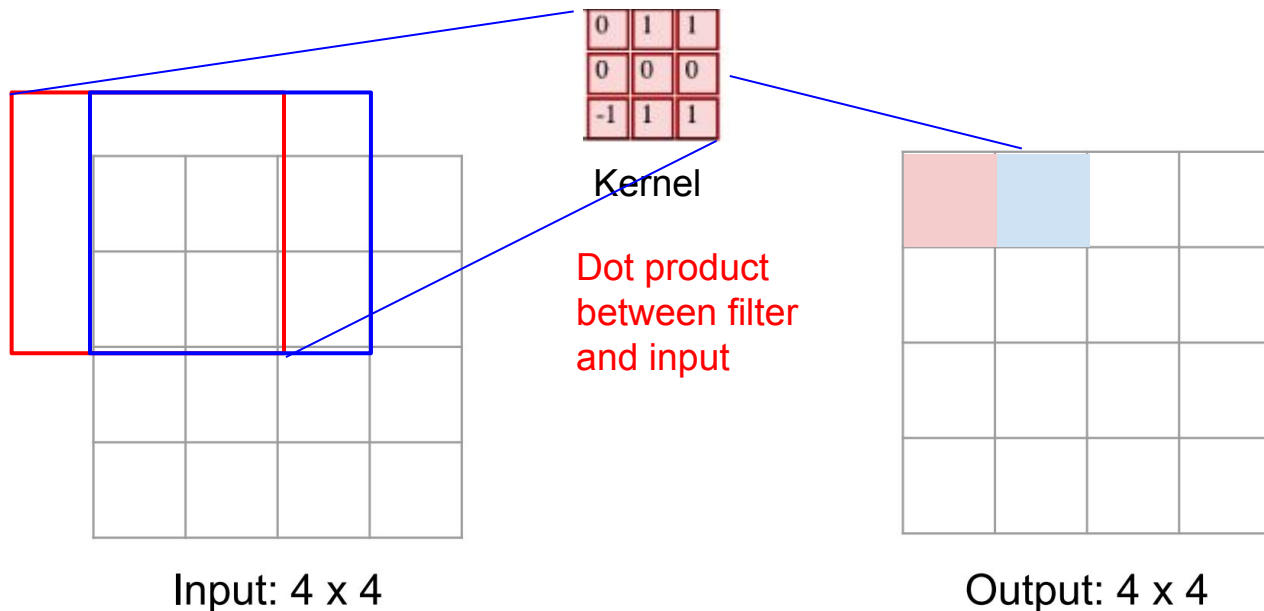
- Recall - normal convolution
- Kernel (3 x 3), stride=1, pad=1



Dense Prediction - Semantic Segmentation

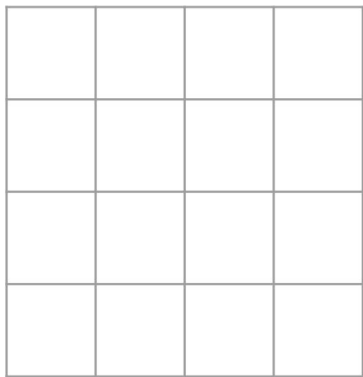
- Upsampling: Transposed Convolution

- Recall - normal convolution
- Kernel (3 x 3), stride=1, pad=1



Dense Prediction - Semantic Segmentation

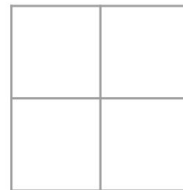
- Upsampling: Transposed Convolution
 - Recall - normal convolution
 - Kernel (3 x 3), **stride=2**, pad=1
 - Filter moves 2 pixels in the input for every one pixel in the output
 - Output is “downsampled” using learnable weights



Input: 4 x 4

0	1	1
0	0	0
-1	1	1

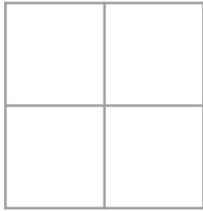
Kernel



Output: 2 x 2

Dense Prediction - Semantic Segmentation

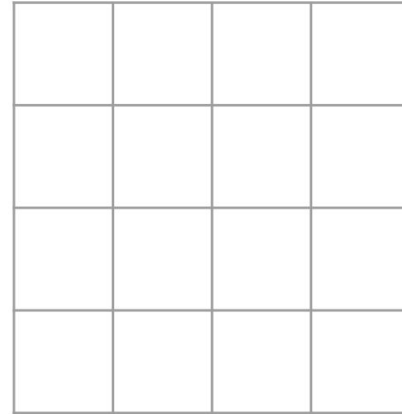
- Upsampling: Transposed Convolution
 - (3 x 3) **transposed convolution**, stride=2, pad=1: Output is upsampled!



Input: 2 x 2

0	1	1
0	0	0
-1	1	1

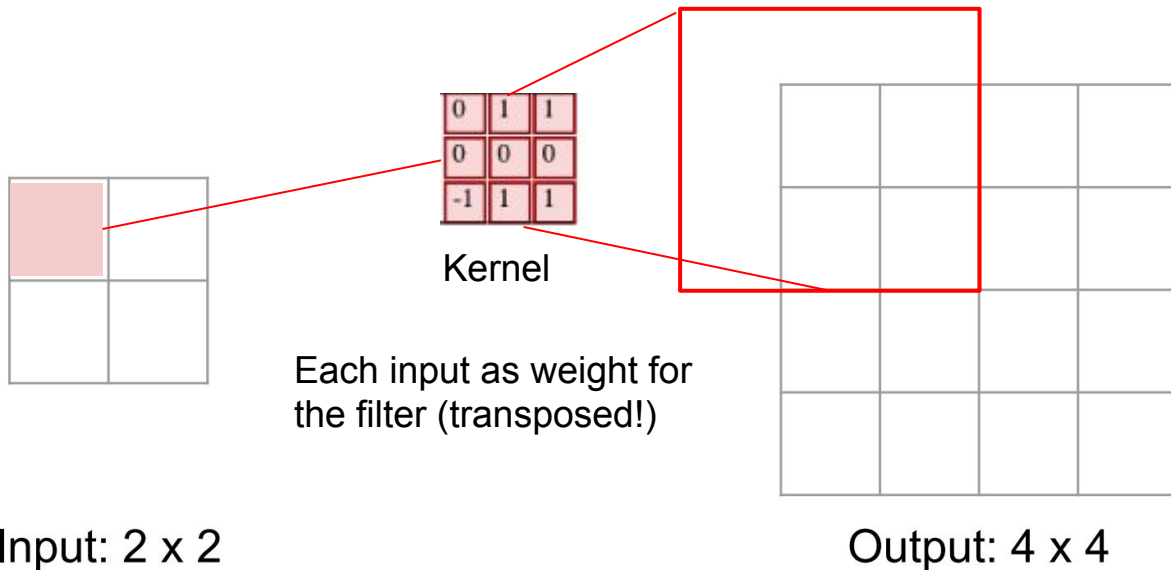
Kernel



Output: 4 x 4

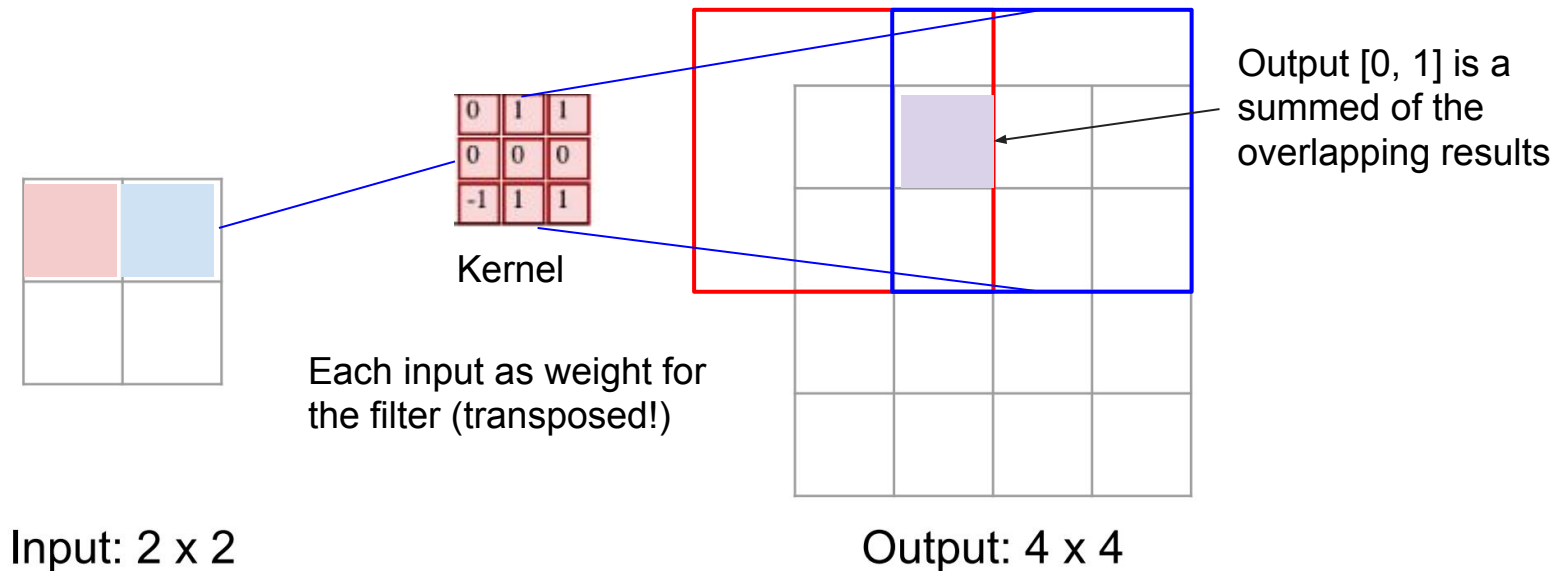
Dense Prediction - Semantic Segmentation

- Upsampling: Transposed Convolution
 - (3 x 3) **transposed convolution**, stride=2, pad=1



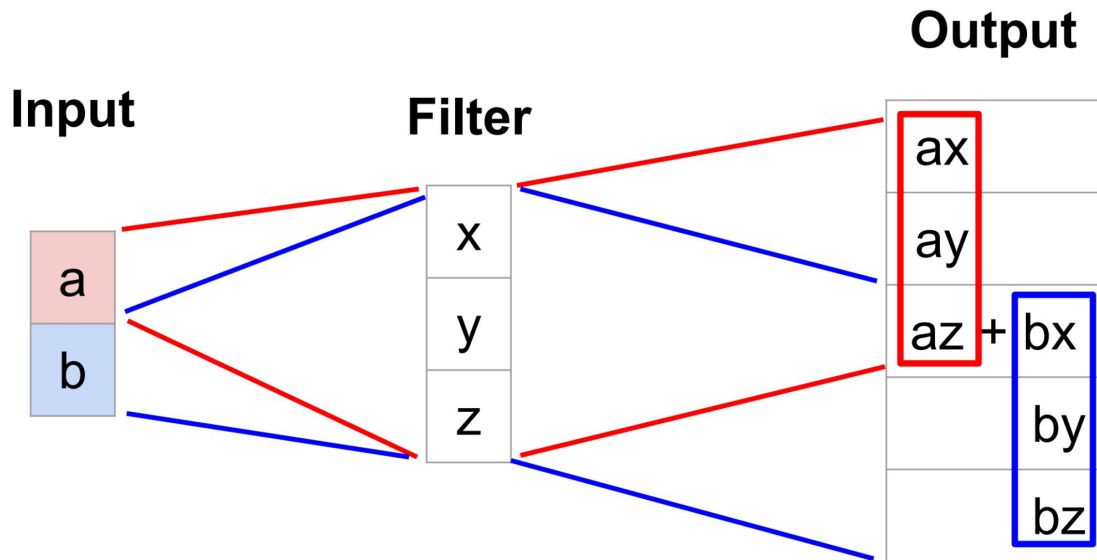
Dense Prediction - Semantic Segmentation

- Upsampling: Transposed Convolution
 - (3 x 3) **transposed convolution**, stride=2, pad=1



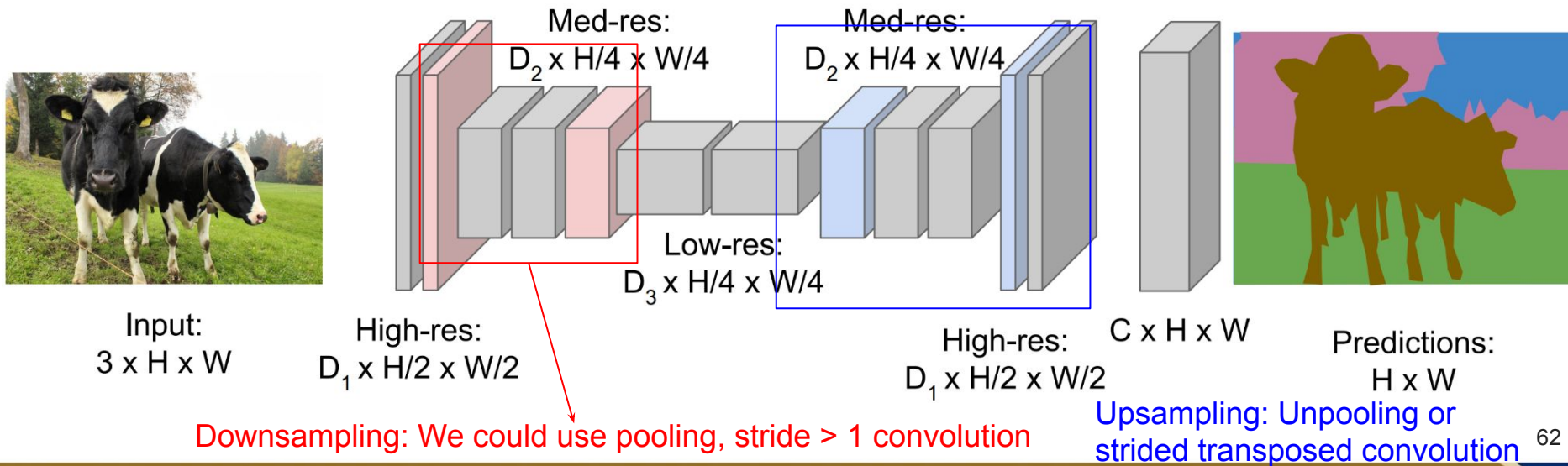
Dense Prediction - Semantic Segmentation

- Upsampling: Transposed Convolution
 - 1D example
 - Output contains copies of the filter weighted by the input, summing at where at overlaps in the output



Dense Prediction - Semantic Segmentation

- Idea: convolutional network
 - We see that convolution can get same-sized outputs
 - Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!

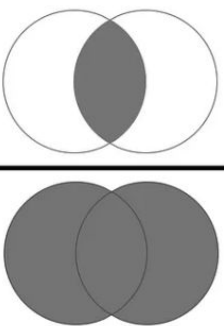


Dense Prediction - Semantic Segmentation

- Evaluation

- Datasets: Cityscapes, ADE20K
- Metric: m-IOU (mean Intersection over Union)
 - The mIoU is the average between the IoU of the segmented objects over all the images of the test dataset.

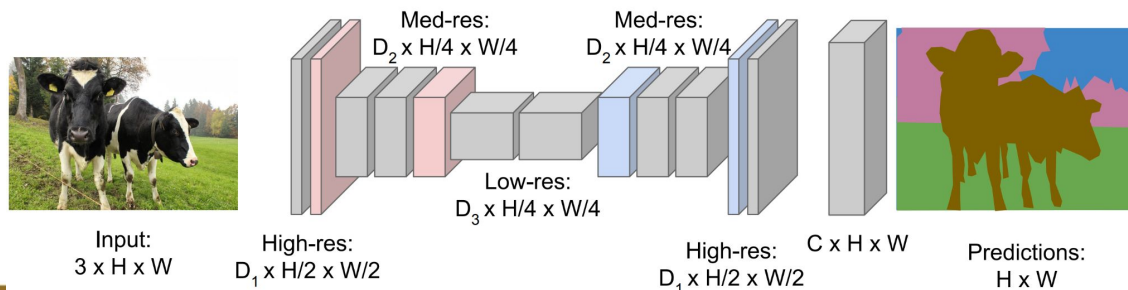


$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$
The equation is accompanied by two Venn diagrams. The top diagram shows two overlapping circles, with the intersection (the area where they overlap) shaded in dark gray, representing the 'Area of Overlap'. The bottom diagram shows two overlapping circles, with the entire area covered by both circles shaded in dark gray, representing the 'Area of Union'.

Dense Prediction - Semantic Segmentation

- Summary

- Task: Label each pixel in the image with a category label
- Design: convolution **encoder** and deconvolution **decoder**
 - Deconvolution: ML people don't like this name
 - More precisely - transposed convolution
 - And we could also use unpooling
- Evaluation: m-IOU
- Tools
 - Deeplab

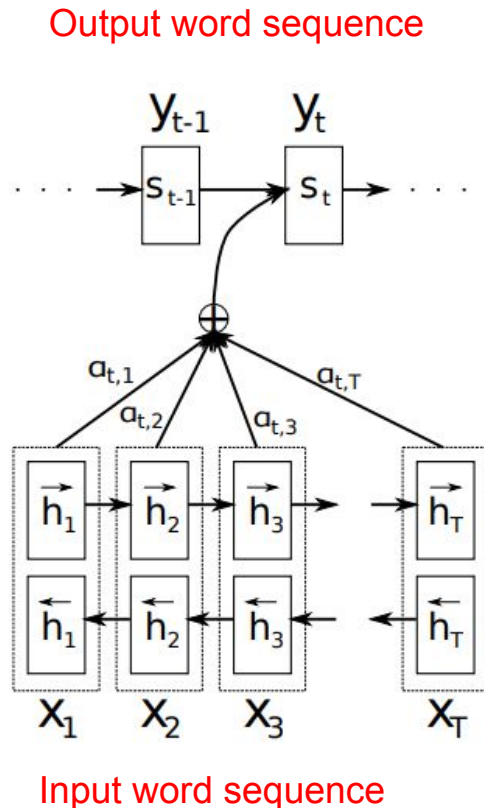


Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Key models
 - CNN
 - Dense prediction
 - Attention model
 - Large-scale vision-language learning
 - Geometric computer vision
 - Bird-Eye-View perception
 - Ego-View perception

Attention Model - Classic

- Translation - learned alignment between words
 - Given source sentence $(x_1, x_2, \dots, x_T)$
 - and previous generated words
 - Decoder: Generate the next word y_t
 - Learned attention (a.k.a alignment) over each source word when generating the t-th target word
 - Intuitively, decoder learns **which source words to pay attention to**
 - because not all the inputs would be used in generating the corresponding output



Attention Model - Classic

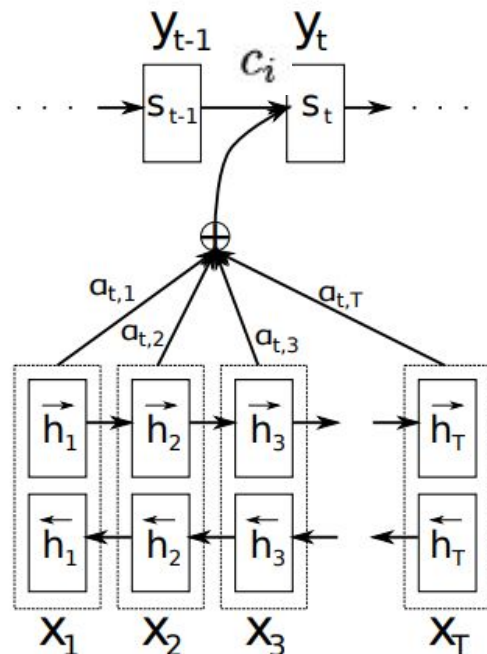
- Translation - learned alignment between words
 - The attention weights are calculated by normalizing the output score of a feed-forward neural network described by the function a that captures the alignment between input at j and output at i .
 - The context vector c_i for the output word y_i is generated using the weighted sum of the attention and hidden states

$$e_{ij} = a(s_{i-1}, h_j) \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})} \quad c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

Last output state

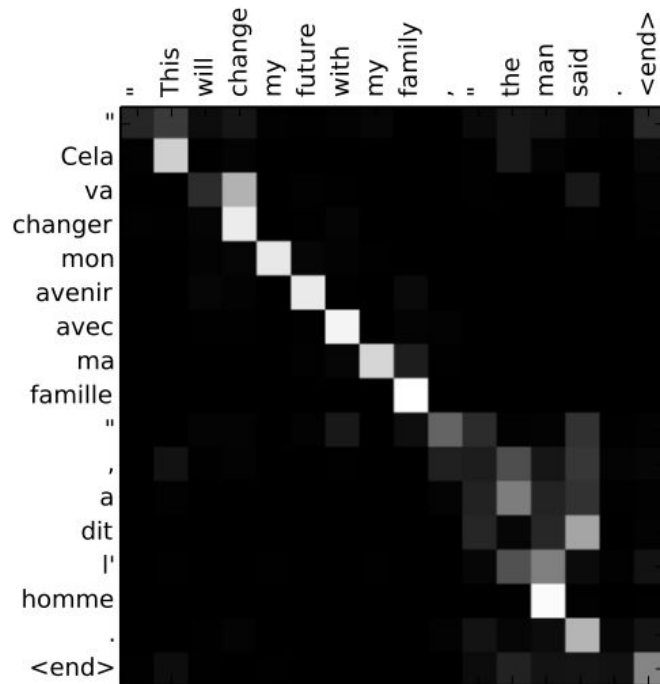
Hidden state from
LSTM/RNN at position j

*Neural Machine Translation by Jointly Learning to Align and Translate



Attention Model - Classic

- Translation - learned alignment between words
 - Visualization
 - Aligning each source word to target word
 - Almost a 1-to-1 alignment for



(d)

Attention Model - Classic

- Visual Question Answering - aligning image part and questions

$$\tilde{v}_I = \sum_i p_i v_i$$

Part-of-image
Attention: weights of Part-of-image, computed based on question and image parts

$v_i \in \mathbf{R}^d$
 $v_I \in \mathbf{R}^{d \times m}$

$$u = \tilde{v}_I + v_Q$$

The encoder vector: question + attended image info

$$p_{ans} = softmax(W_a u)$$



Figure 2: CNN based image model

m=14 x 14



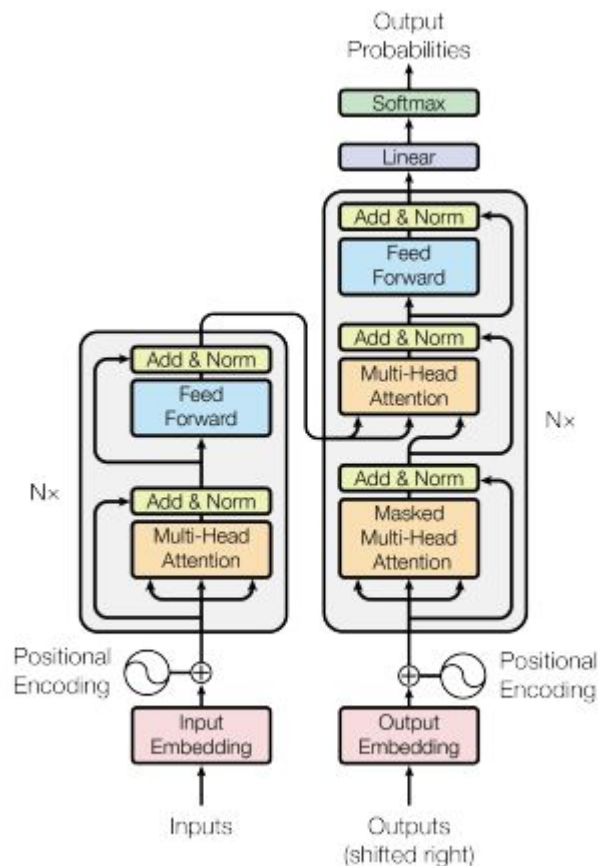
What is the mustache made of?

*Bias vector is not shown for simplicity

[1] Stacked Attention Networks for Image Question Answering. CVPR 2016

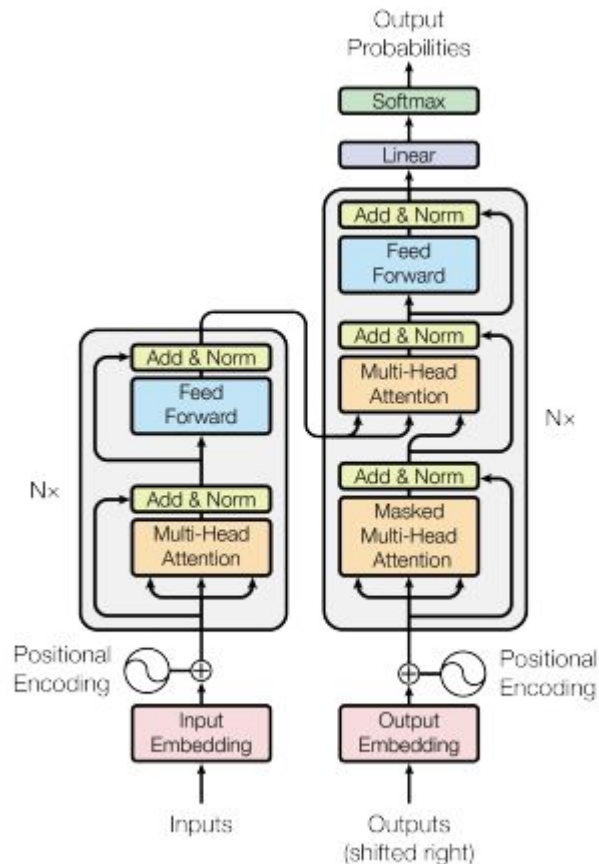
Attention Model - Self-Attention

- Translation - learned alignment between words
 - **Replacing the LSTMs / RNNs with attention layers**
 - Vanilla RNNs are Slow and have terrible long-term memory
 - LSTMs and GRUs fix the memory problem, but are still slow and sequential



Attention Model - Self-Attention

- Translation - learned alignment between words
 - Replacing the LSTMs / RNNs with attention layers
 - Self-attention and cross-attention
 - **Query, Key, Value in attention**
 - Query: This is what pays the attention
 - Keys: These help queries figure out how much attention to pay to each of the values
 - Attention weights are **query-key** similarities
 - Values: These are paid attention to
 - So `computed_attention x values`



Attention Model - Self-Attention

Suppose you have N token inputs, computing N attention weights

1. Attention weights $a_{1:N}$ are **query-key** similarities:

$$\hat{a}_i = q \cdot k_i$$

Normalized via softmax

So given one query, get attention for each key.

2. **Output z** is attention-weighted average of values $v_{1:N}$:

$$z = \sum_i \hat{a}_i v_i = \hat{a} \cdot v$$

3. Usually, k and v are derived from the same input x :

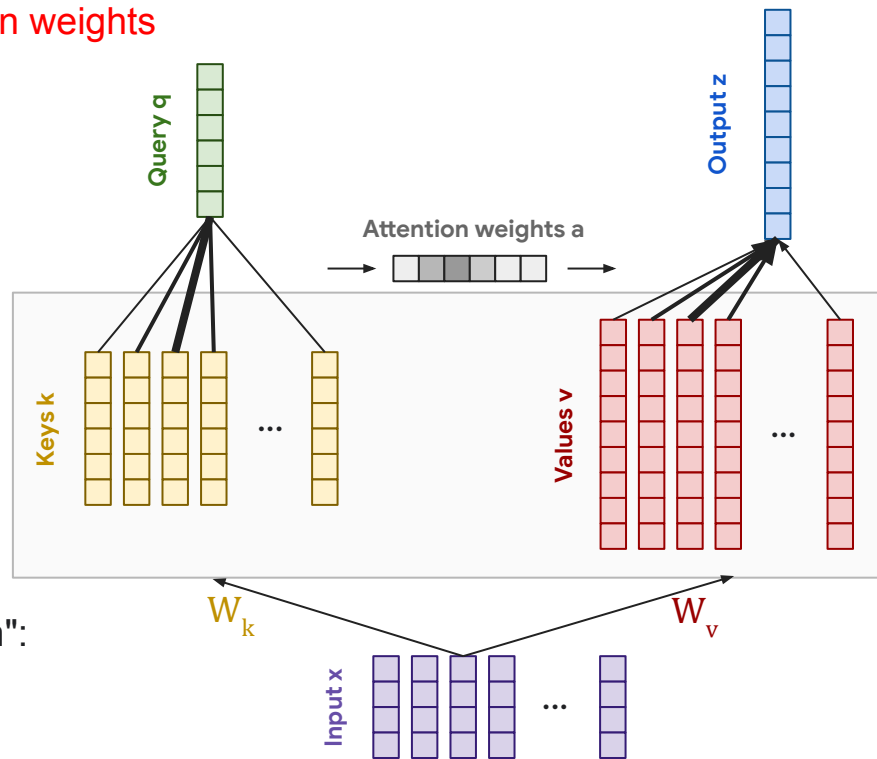
$$k = W_k \cdot x \quad v = W_v \cdot x$$

The query q can come from a separate input y :

$$q = W_q \cdot y \text{ "Cross attention"}$$

Or from the same input x ! Then we call it "self attention":

$$q = W_q \cdot x$$

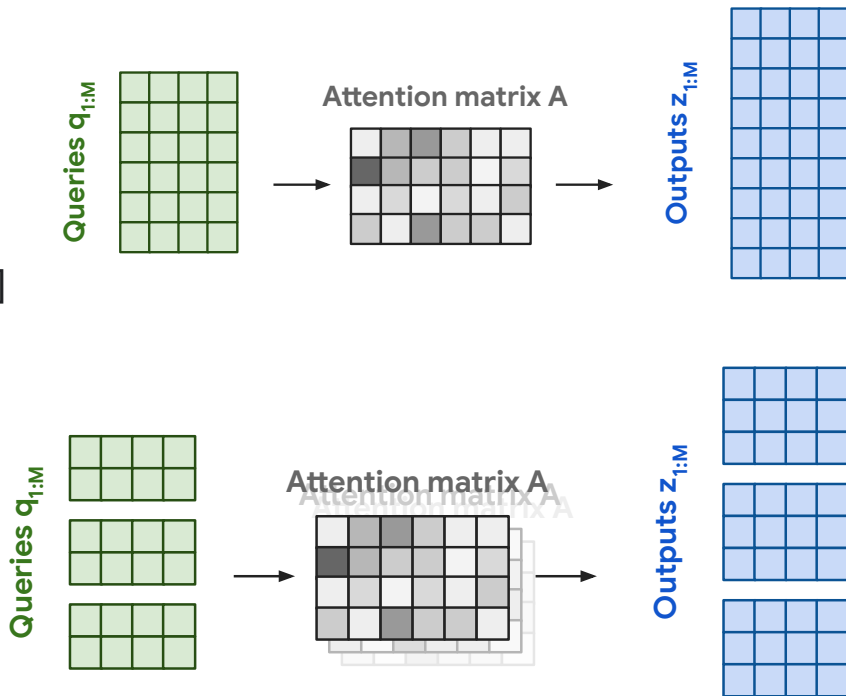


Attention Model - Self-Attention

But that's not actually it! There are a few more details:

1. We usually use **many queries** $q_{1:M}$, not just one. Stacking them leads to the Attention matrix $A_{1:N,1:M}$ and subsequently to many outputs: $z_{1:M} = \text{Attn}(q_{1:M}, x) = [\text{Attn}(q_1, x) \mid \text{Attn}(q_2, x) \mid \dots \mid \text{Attn}(q_M, x)]$
2. We usually use "**multi-head**" attention. This means the operation is repeated K times and the results are concatenated along the feature dimension. **Ws** differ.
3. The most commonly seen formulation:

$$Z = \text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V$$



Note that the complexity is $O(N^2)$, N is number of tokens

Attention Model - Vision Transformer

- Applying the transformer model for vision
 - Representing image pixels as **sequences**
 - But remember the complexity is $O(N^2)$

Attention Model - Vision Transformer

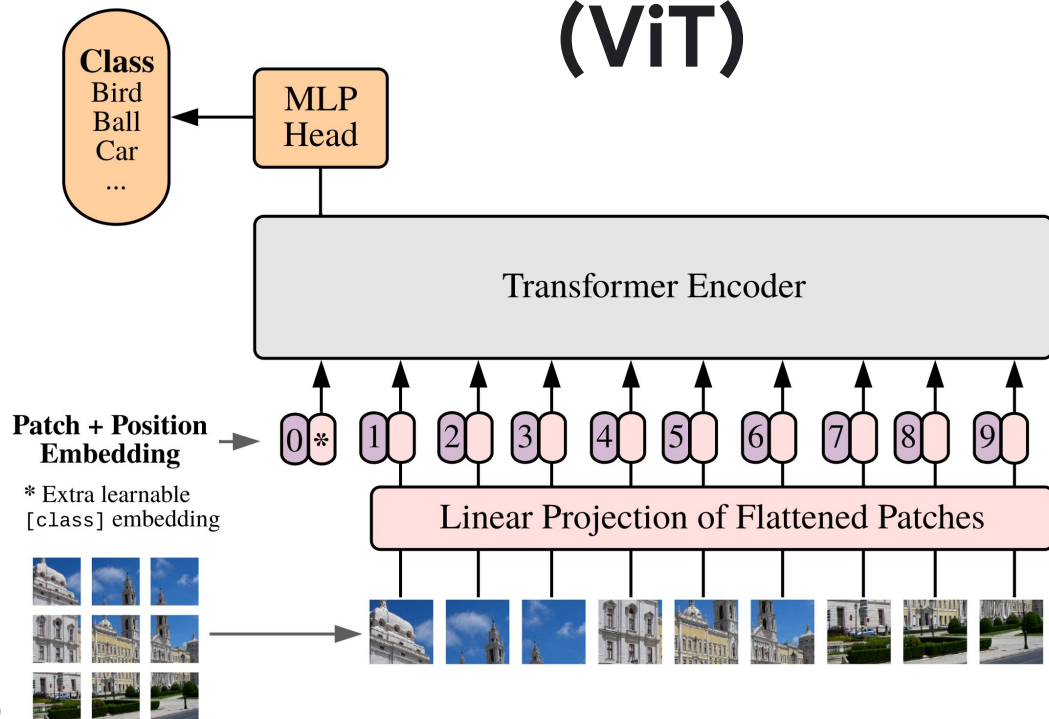
Vision Transformer (ViT)

Many prior works attempted to introduce self-attention at the pixel level.

For 224px^2 , that's 50k sequence length, too much!

Thus, most works restrict attention to local pixel neighborhoods, or as high-level mechanism on top of detections.

The **key breakthrough** in using the full Transformer architecture, standalone, was to **"tokenize" the image by cutting it into patches** of 16px^2 , and treating each patch as a token, e.g. embedding it into input space.



Attention Model - Vision Transformer

- Summary
Visualization



Summary

- CNNs vs Vision Transformers

- Key design different

- Convolution is translation invariant* (this is what we call “inductive bias”, a property that is built into the design)

- An object translated should has the same object response on the feature maps

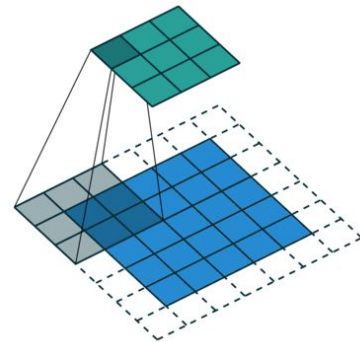
- While attention is permutation invariant

- Change the order of the element the attention stays the same
 - The inductive bias is more relaxed

- Therefore

- Given more data, transformers are better than CNNs

- Another perk - transformers can be used for multimodal data



* And also translation equivariance

Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Key models
 - CNN
 - Dense prediction
 - Attention model
 - Large-scale vision-language learning
 - Geometric computer vision
 - Bird-Eye-View perception
 - Ego-View perception

Large-scale Vision-Language Learning

- CLIP (Contrastive Language–Image Pre-training) by OpenAI in 2021
 - Train on 400 million image+text crawled from the internet
 - Proved that large-scale (noisy) vision-language pre-training can lead to great performance (with proper prompt engineering)
 - Any zero-shot downstream classification tasks
 - AI generated content (text-to-image generation)
 - DALL-E, etc.

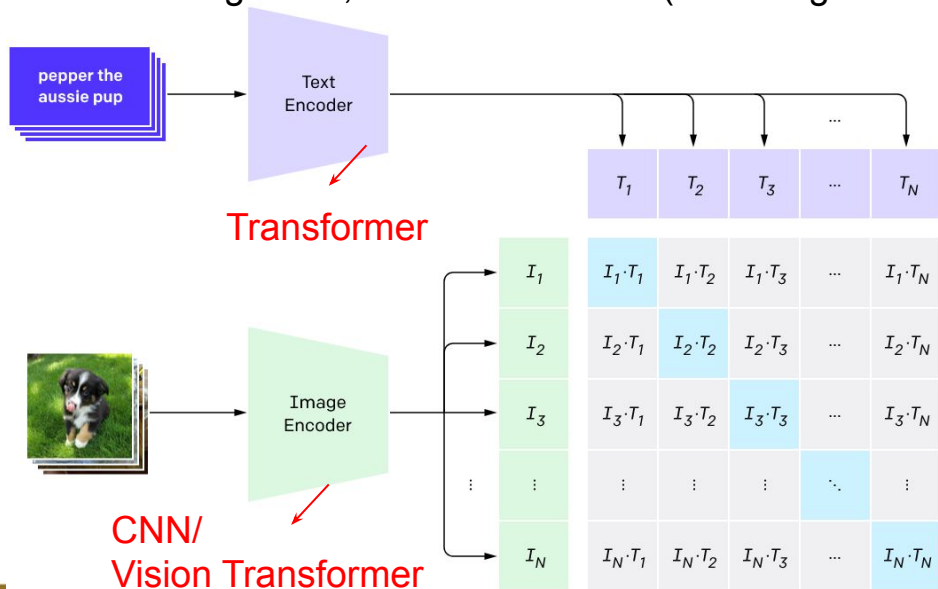
Large-scale Vision-Language Learning

- CLIP (Contrastive Language–Image Pre-training) by OpenAI in 2021
 - Train on 400 million image+text crawled from the internet
 - Proved that large-scale (noisy) vision-language pre-training can lead to great performance (with proper prompt engineering)
 - Any zero-shot downstream classification tasks
 - AI generated content (text-to-image generation)
 - DALL-E, etc.
- Why is this useful for Embodied AI?
 - We need to generalize our perception model to just recognizing discrete object classes
 - CIFAR-10/100, ImageNet-1K, ImageNet-21K
 - Finite set of vocabularies



Large-scale Vision-Language Learning

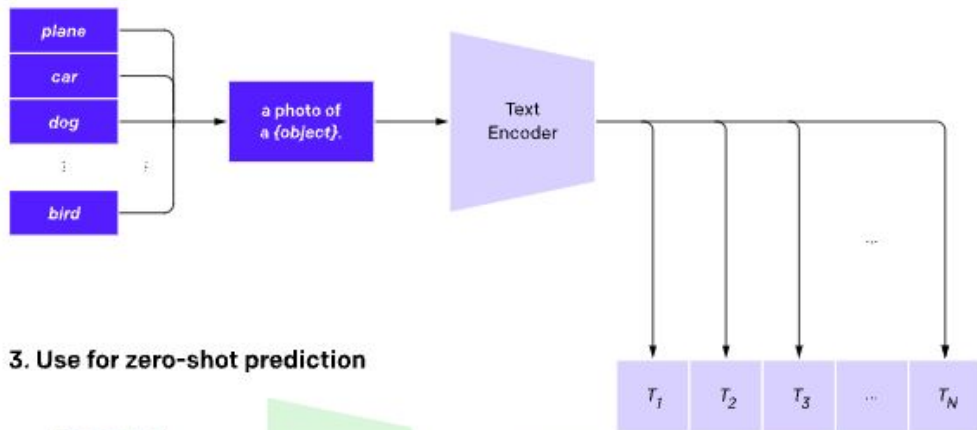
- CLIP (Contrastive Language–Image Pre-training) by OpenAI in 2021
 - Two Key Designs
 - Contrastive pre-training
 - Positive image-text + random negatives, train with CE loss (need large batch size to work - 32k)



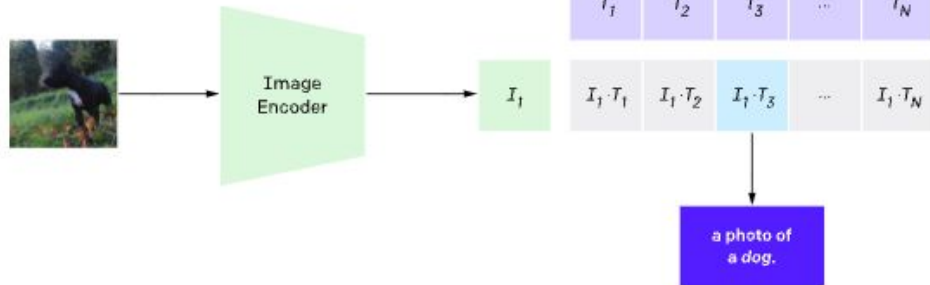
Large-scale Vision-Language Learning

- CLIP (Contrastive Language–Image Pre-training) by OpenAI in 2021
 - Two Key Designs
 - Contrastive pre-training
 - Prompt Engineering
 - “A photo of [class]”
 - Simply add prefix text
 - Improves zero-shot

2. Create dataset classifier from label text



3. Use for zero-shot prediction



Agenda

- Objectives

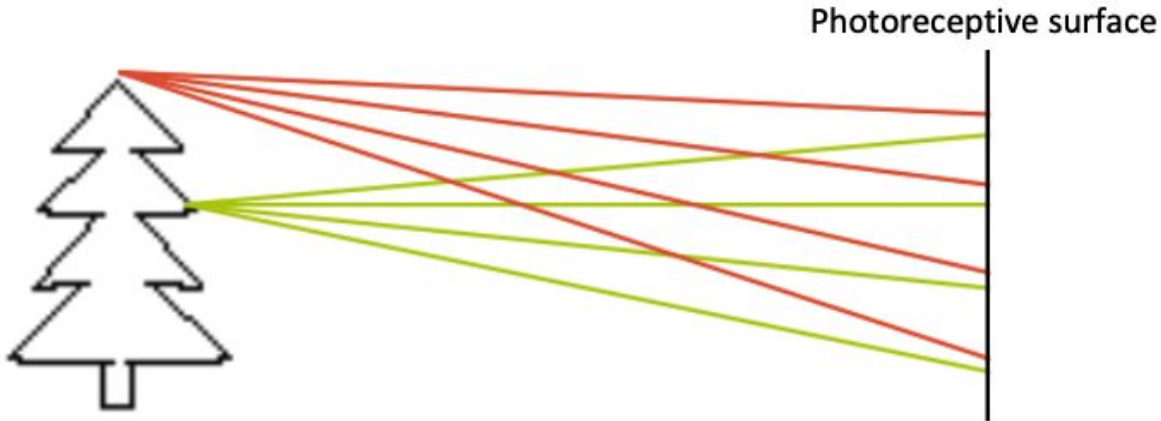
- We will learn the key perception models of Embodied AI

- The tasks

- Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Geometric computer vision
 - Camera models
 - Perspective projection
 - Camera calibration
 - Bird-Eye-View perception
 - Ego-View perception

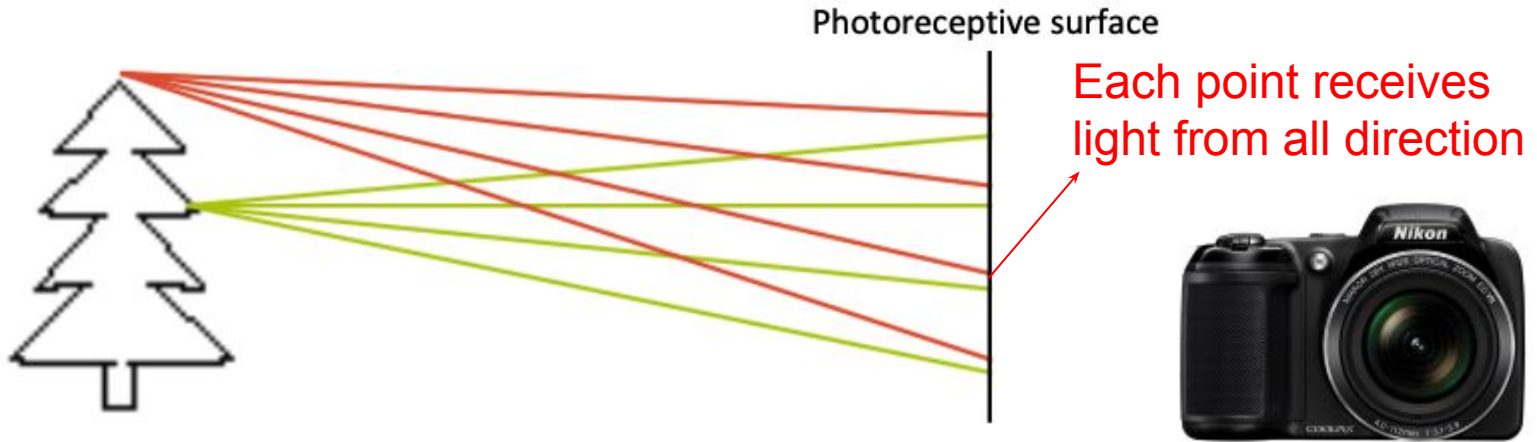
Cameras

- How to capture an image of the world
 - (Digital) Cameras -> capture light -> converted to digital image
 - Light is reflected by the object and scattered in all directions
 - if we simply add a photoreceptive surface, the captured image will be extremely blurred
 - why?



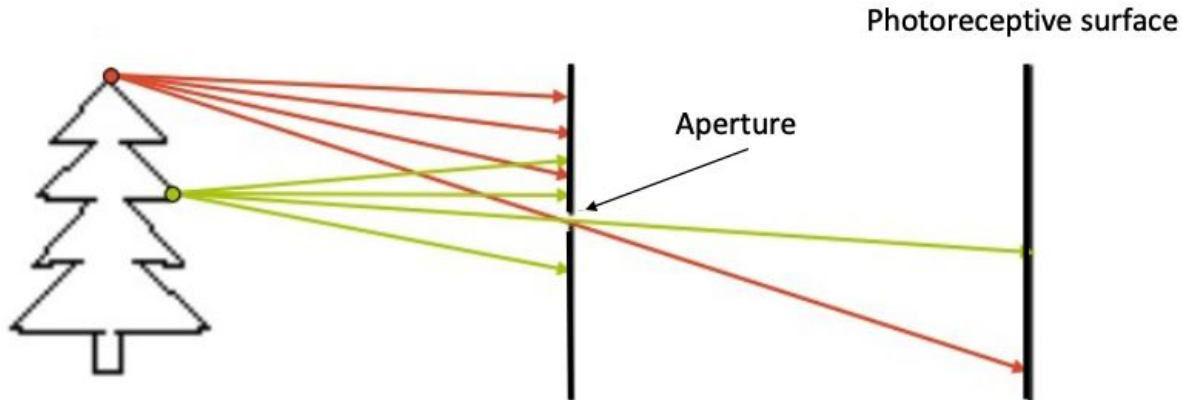
Cameras

- How to capture an image of the world
 - (Digital) Cameras -> capture light -> converted to digital image
 - Light is reflected by the object and scattered in all directions
 - if we simply add a photoreceptive surface, the captured image will be extremely blurred
 - why?



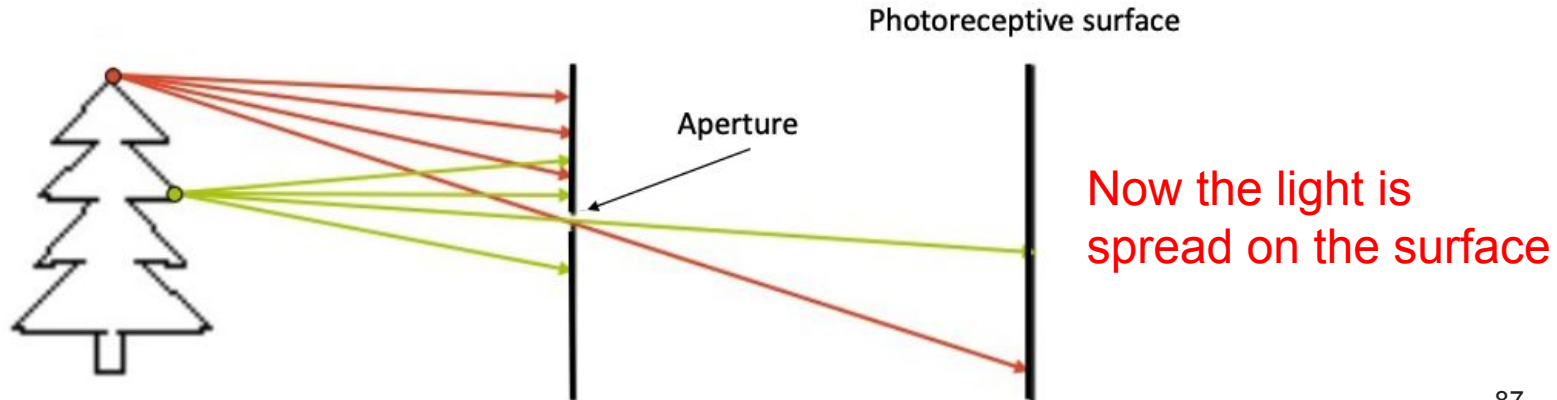
Pinhole Camera

- How to capture an image of the world
 - Add a barrier to block off most of the light/rays
 - The light can only come through the aperture in the center
 - Pinhole camera: a camera without a lens but with a tiny aperture, a pinhole



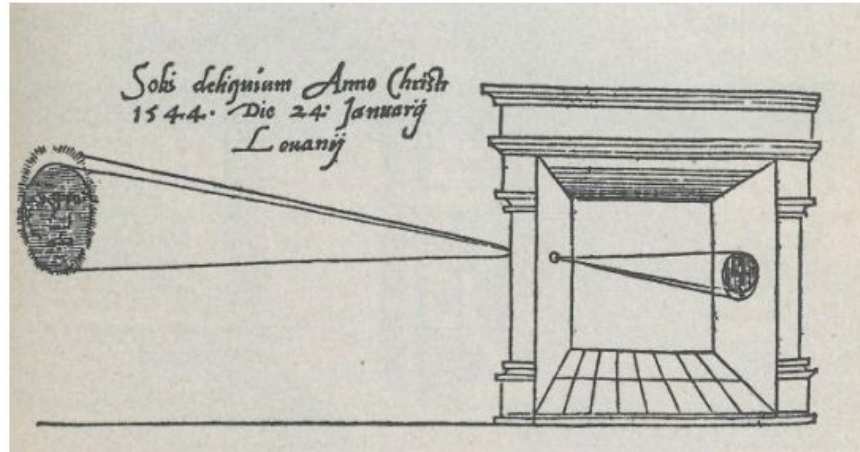
Pinhole Camera

- How to capture an image of the world
 - Add a barrier to block off most of the light/rays
 - The light can only come through the aperture in the center
 - Pinhole camera: a camera without a lens but with a tiny aperture, a pinhole

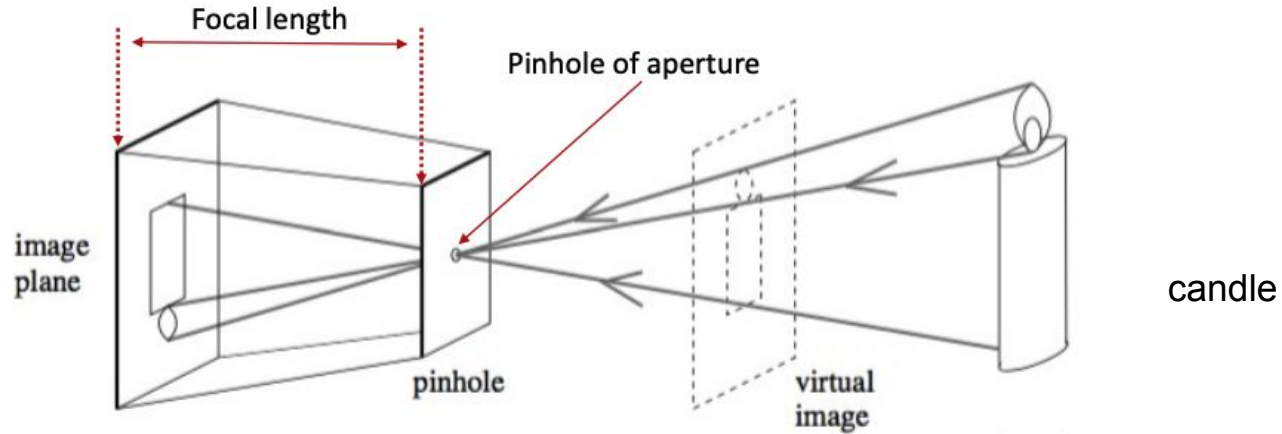


Pinhole Camera

- A long history
 - First clear description from Leonardo Da Vinci (1502)
 - Oldest known published drawing of a camera obscura by Gemma Frisius (1544)



Pinhole Camera



- Perspective projection creates *inverted images*
- Sometimes it is convenient to consider a virtual image associated with a plane lying in front of the pinhole
- Virtual image is equivalent to the actual one except for on scale

Pink



- How to project a point in the world to the image plane

Suppose we know P in the camera frame

$$P_c = (X_c, Y_c, Z_c)$$

Perspective

$$p = (x, y, z)$$

How to get p on the image plane
(3D, O is the origin)



Pink



Perspective

$$p = (x, y, z)$$

- Since P, O, and p are collinear (on the same line): $\overrightarrow{Op} = \lambda \overrightarrow{OP}$

$$\begin{cases} x = \lambda X \\ y = \lambda Y \\ z = \lambda Z \end{cases} \Leftrightarrow \lambda = \frac{x}{X} = \frac{y}{Y} = \frac{z}{Z} \Rightarrow \begin{cases} x = f \frac{X}{Z} \\ y = f \frac{Y}{Z} \end{cases} \quad \text{C}$$

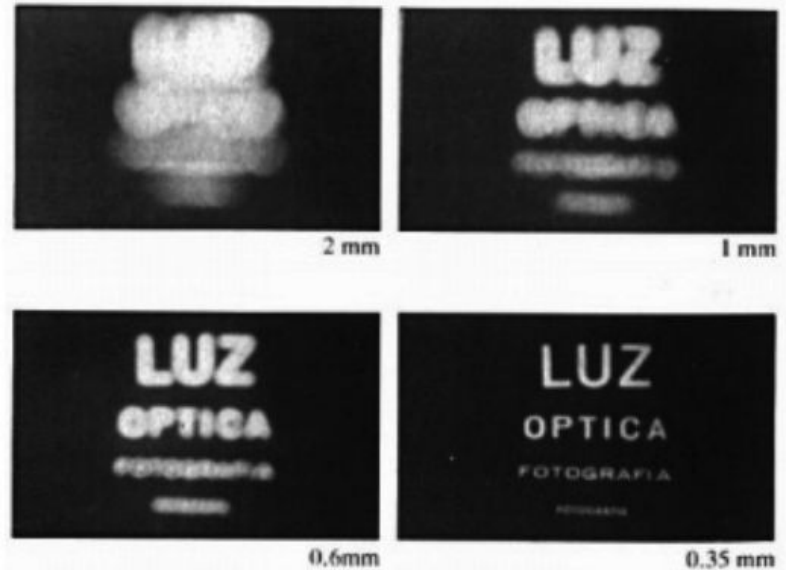
Focal Length

$$z = f$$

So if we know the focal length, we know the 3D coordinates of p. Will use this conclusion later.

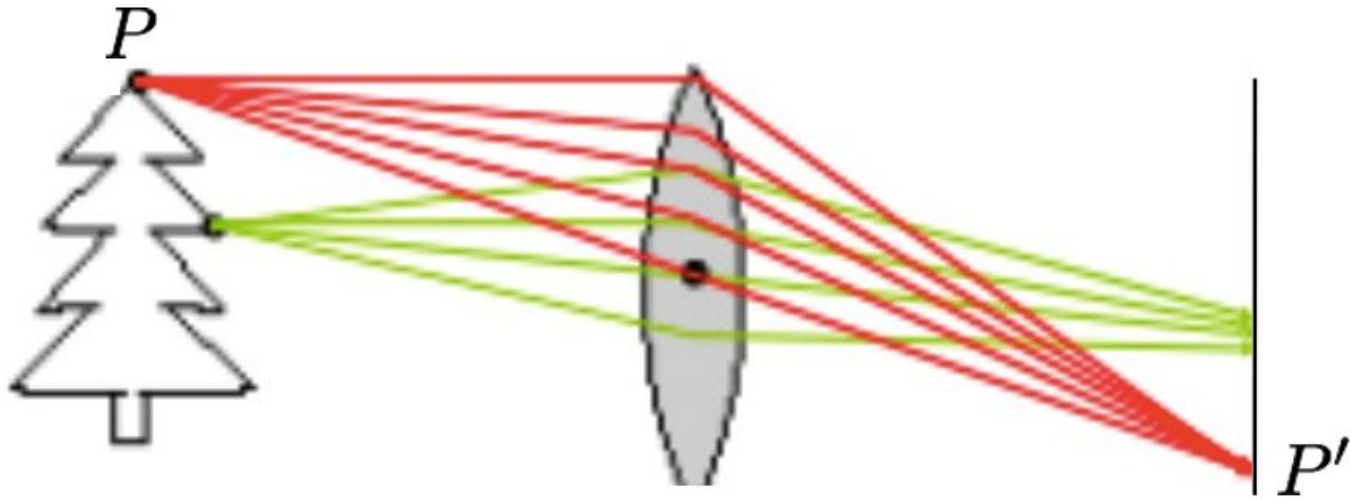
Issues with Pinhole Camera

- Larger hole: more lights passing through the pinhole -> blurry
- Smaller: darker
- Solution: add a lens to replace the pinhole

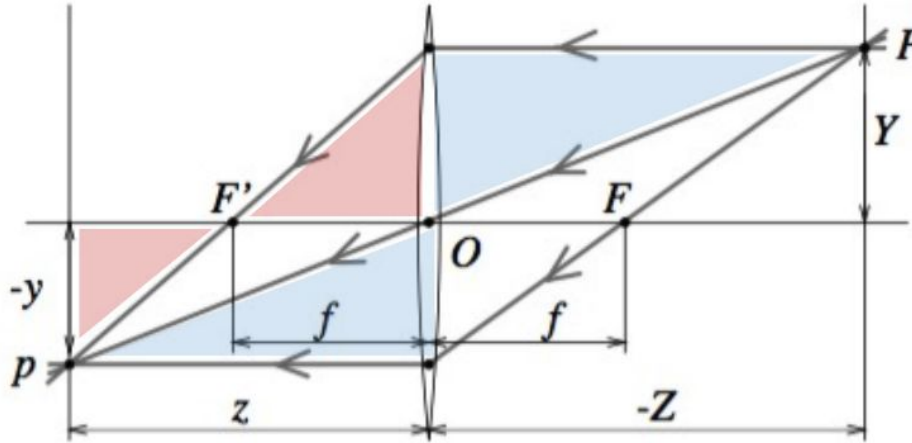


Lenses

- Lens: an optical element (glass) that focuses light by means of refraction



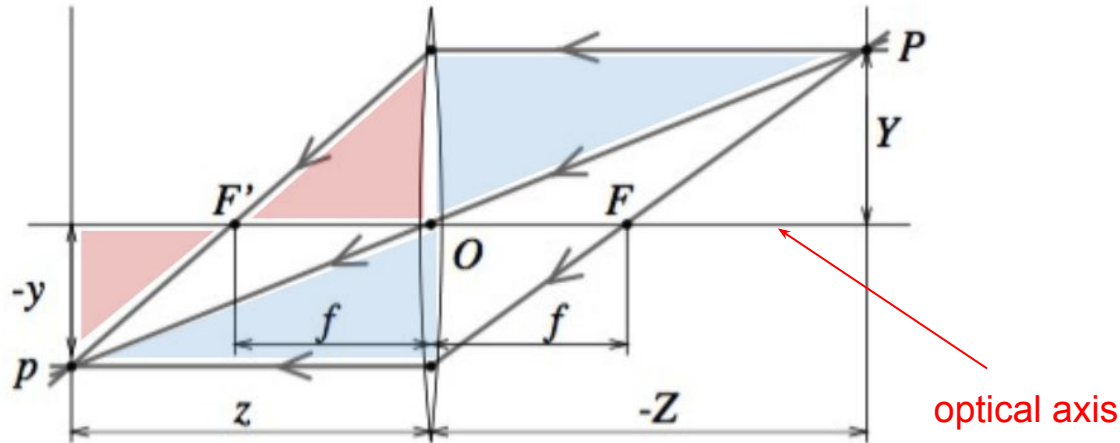
Thin Lens Model



- Key properties

- Rays passing through O (lens center) are not refracted
- Rays parallel to the optical axis are focused on the focal point F'
- All rays passing through P are focused by the lens on the point p

Thin Lens Model



So p receives more rays from P compared to pinhole

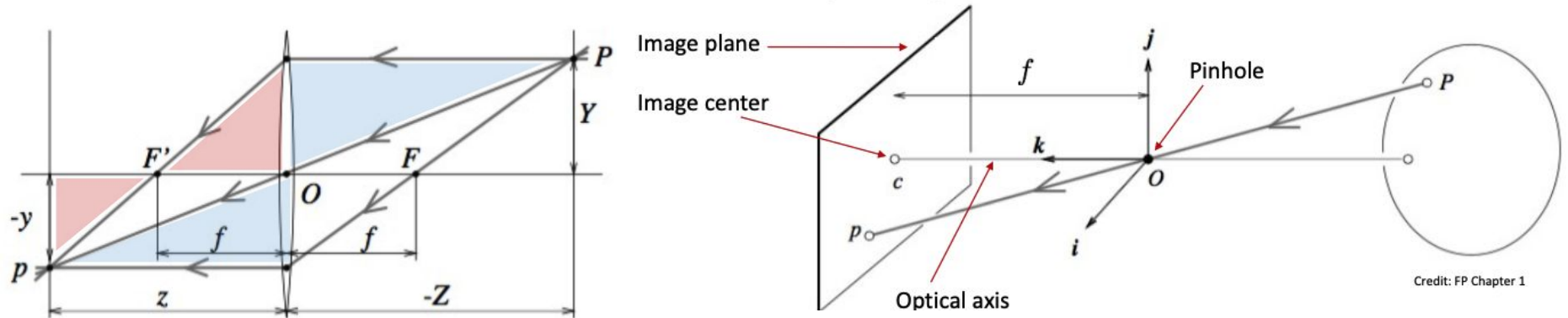
- Key properties

- Rays passing through O (lens center) are not refracted
- Rays parallel to the optical axis are focused on the focal point F'
- All rays passing through P are focused by the lens on the point p

Thin Lens Model

- Notes

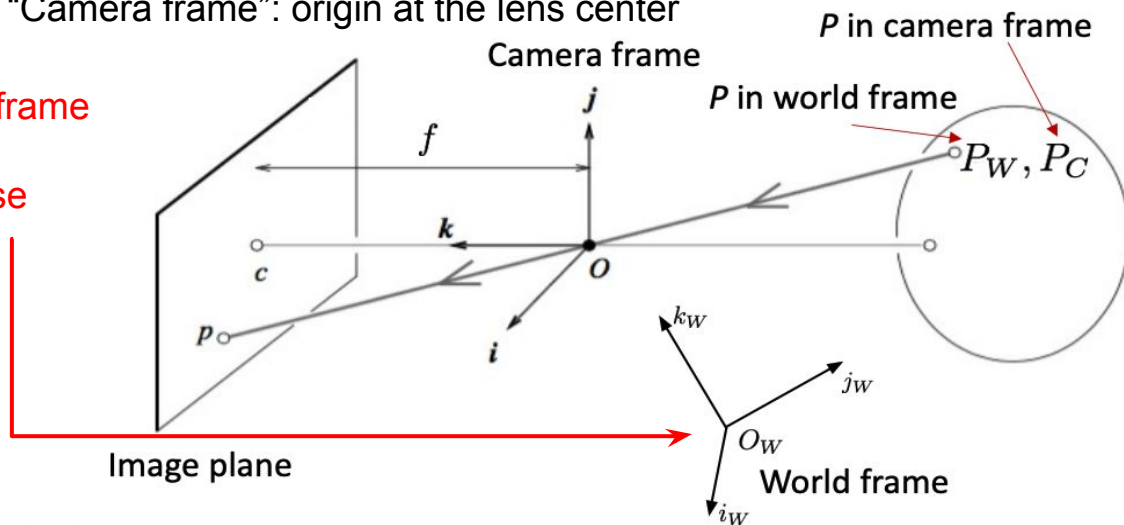
- If we consider z as the focal length, the perspective transformation equation is the same between thin lens model and pinhole model
- We ignore the depth of field effect for the thin lens model for simplicity
 - Assuming camera is focused at infinite far



Perspective Projection

- Goal: project world points onto the camera image
- Assumption: we have a pinhole camera model (all results also hold under thin lens model, assuming camera is focused at ∞)
 - “frame”: coordinate system
 - “Camera frame”: origin at the lens center

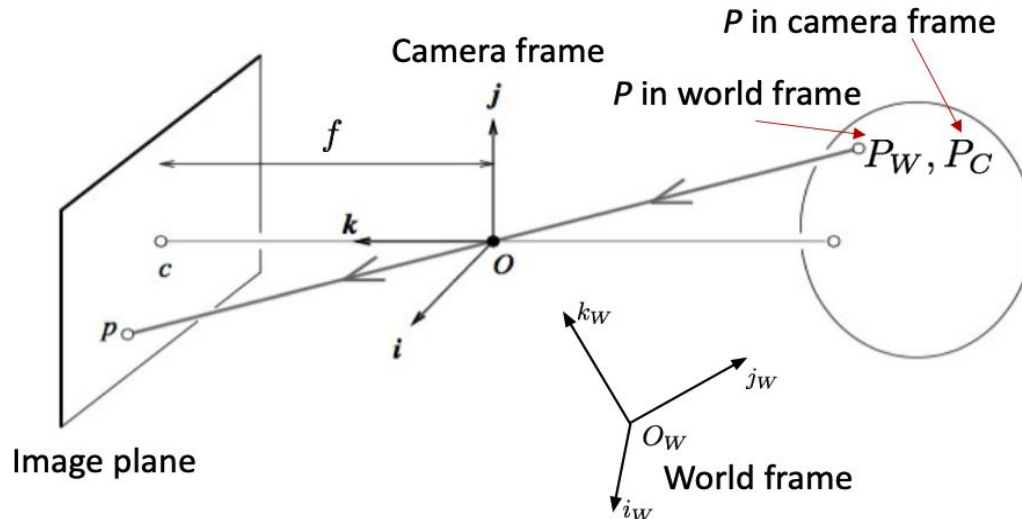
And the world frame
has an origin
somewhere else



Perspective Projection

- Steps

- 1. Map P_c into p (image plane)
- 2. Map p into (u, v) pixel coordinates
- 3. Transform P_w into P_c



P_w and P_c are the same point, but in different frame (coordinate system) with different origins



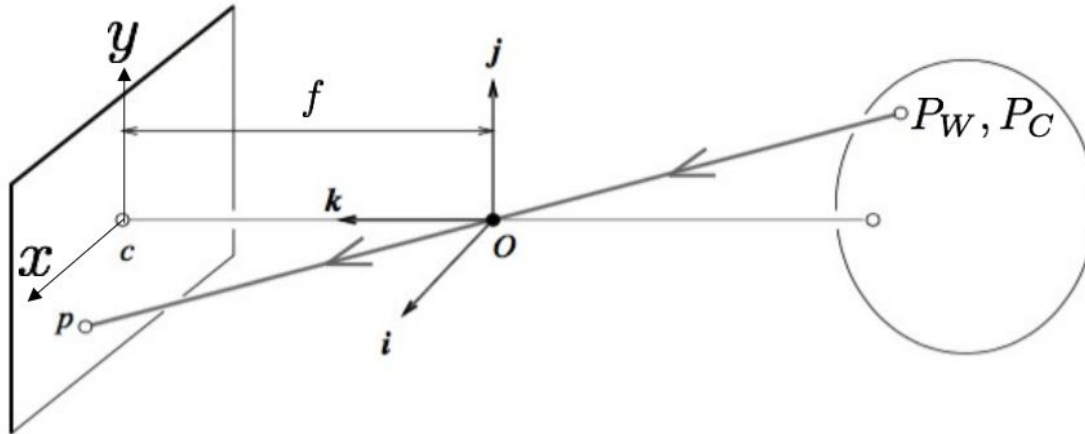
Perspective Projection

- Steps

- 1. Map $P_c = (X_c, Y_c, Z_c)$ into $p = (x, y)$ (image plane)
- We already have

$$\begin{cases} x = f \frac{X_c}{Z_c} \\ y = f \frac{Y_c}{Z_c} \end{cases}$$

why?

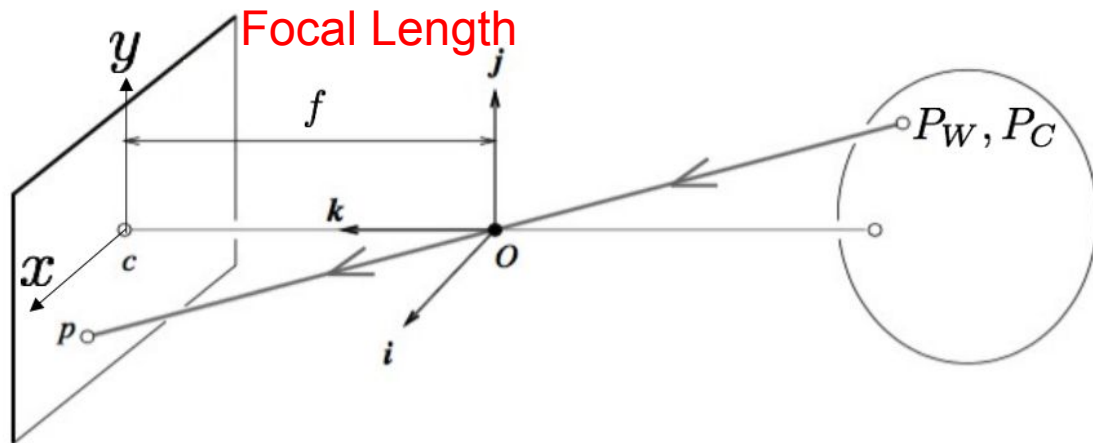


Perspective Projection

- Steps

- 1. Map $P_c = (X_C, Y_C, Z_C)$ into $p = (x, y)$ (image plane)
- We already have

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases}$$



Remember P, O, p are collinear (on the same line)

$$\overline{Op} = \lambda \overline{OP}$$

$$\begin{cases} x = \lambda X_C \\ y = \lambda Y_C \\ z = \lambda Z_C \end{cases}$$

$$z = f$$

$$\lambda = \frac{x}{X_C} = \frac{y}{Y_C} = \frac{z}{Z_C}$$

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases}$$



Perspective Projection

- Steps

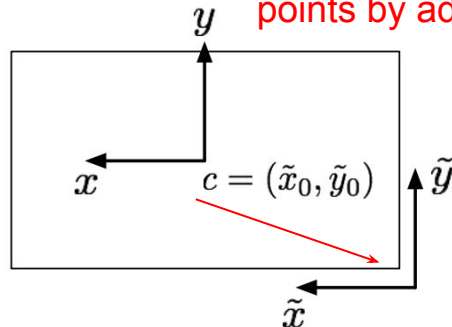
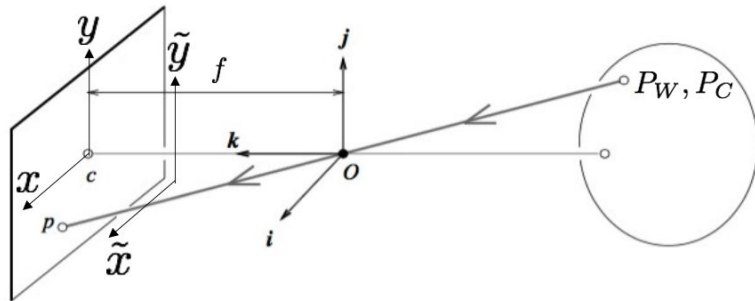
- 2. Map $p = (x, y)$ into (u, v) pixel coordinates
- Convert the coordinates to the corners of the image plane**

These x, y are still represented using coordinates in the camera frame

We want x, y in the image frame

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases} \quad \tilde{x} = f \frac{X_C}{Z_C} + \tilde{x}_0, \quad \tilde{y} = f \frac{Y_C}{Z_C} + \tilde{y}_0,$$

Suppose c is the known center of the image plane, we can simply translate the points by adding c



Perspective Projection

- Steps

- 2. Map $p = (x, y)$ into (u, v) pixel coordinates
- Convert the image coordinates to **pixel coordinates** (image is in centimeters, etc; pixel is different)
 - Simply change the scale of the coordinates

We already have $\tilde{x} = f \frac{X_C}{Z_C} + \tilde{x}_0, \quad \tilde{y} = f \frac{Y_C}{Z_C} + \tilde{y}_0,$

Number of pixels
per unit distance
in image
coordinates

$$u = k_x \tilde{x} = \underbrace{k_x f}_{\alpha} \frac{X_C}{Z_C} + \underbrace{k_x \tilde{x}_0}_{u_0}$$

$$v = k_y \tilde{y} = \underbrace{k_y f}_{\beta} \frac{Y_C}{Z_C} + \underbrace{k_y \tilde{y}_0}_{v_0}$$

$\Rightarrow$

$$\begin{aligned} u &= \alpha \frac{X_C}{Z_C} + u_0 \\ v &= \beta \frac{Y_C}{Z_C} + v_0 \end{aligned}$$

Nonlinear transformation



Homogeneous coordinates

- We want to represent the transformation as a linear mapping, so we need homogeneous coordinates

From euclidean coordinates to homogeneous coordinates. Basically you have one more dimension

$$\begin{pmatrix} x \\ y \end{pmatrix} \Rightarrow \lambda \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad \begin{pmatrix} x \\ y \\ z \end{pmatrix} \Rightarrow \lambda \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Perspective Projection

- Projection can be equivalently written in homogeneous coordinates

The XYZ in camera frame

Focal length +
Principal point
offset + the
pixel–real-world
scale

$$\begin{matrix} & K & \\ \left[\begin{array}{ccc} \alpha & 0 & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{array} \right] & \begin{array}{c} 0 \\ 0 \\ 0 \end{array} \end{matrix}$$

Camera matrix/
Matrix of intrinsic parameters

$$u = k_x \tilde{x} = \underbrace{k_x f}_{\alpha} \frac{X_C}{Z_C} + \underbrace{k_x \tilde{x}_0}_{u_0}$$

$$v = k_y \tilde{y} = \underbrace{k_y f}_{\beta} \frac{Y_C}{Z_C} + \underbrace{k_y \tilde{y}_0}_{v_0}$$

$$\begin{pmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{pmatrix}$$

P_c in homogeneous
coordinates

$$= \begin{pmatrix} \alpha X_c + u_0 Z_c \\ \beta Y_c + v_0 Z_c \\ Z_c \end{pmatrix}$$

The x,y in pixels

Homogeneous pixel
coordinates

$$p^h = [K \quad 0_{3 \times 1}] P_C^h$$



Perspective Projection

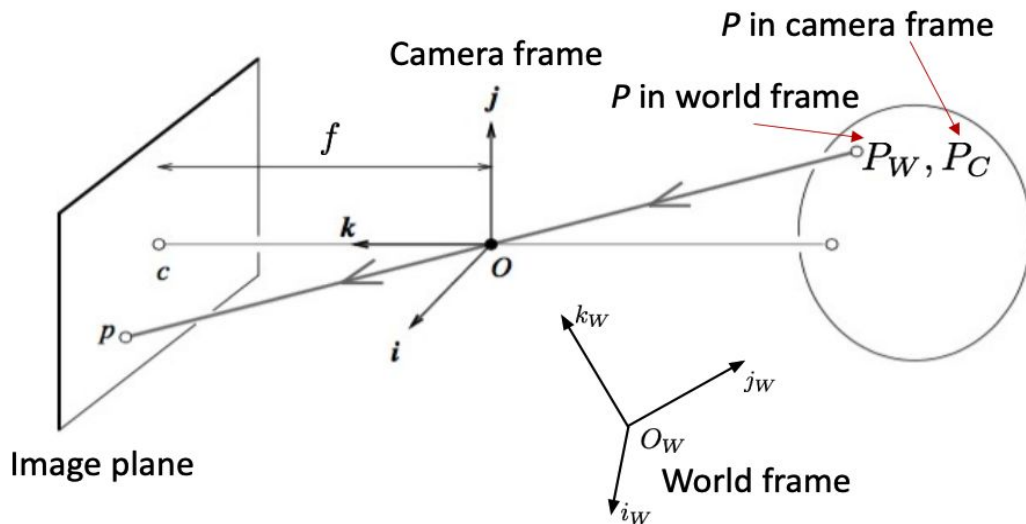
- Projection can be equivalently written in homogeneous coordinates

Point p in homogeneous pixel coordinates $\nearrow p^h = [K \quad 0_{3 \times 1}] P_C^h \nwarrow$ Point P_c in homogeneous camera coordinates

Perspective Projection

- Steps

- 1. Map P_c into p (image plane) ✓
- 2. Map p into (u, v) pixel coordinates ✓
- 3. Transform P_w into P_c

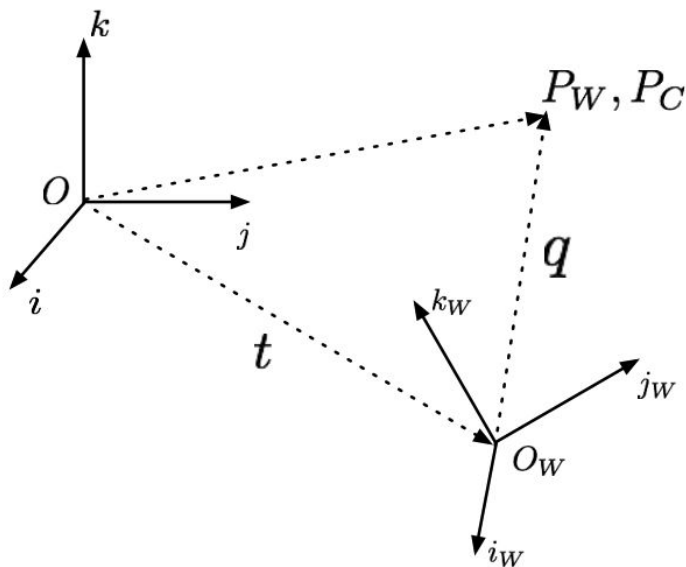


$$p^h = [K \quad 0_{3 \times 1}] P_C^h$$

Perspective Projection

- Steps

- 3. Transform P_W into P_C



$$P_C = t + q$$

Rotation + Translation:
Rigid transformation

$$q = R P_W$$

where R is the rotation matrix
relating camera and world
frames

$$R = \begin{bmatrix} i_W \cdot i & j_W \cdot i & k_W \cdot i \\ i_W \cdot j & j_W \cdot j & k_W \cdot j \\ i_W \cdot k & j_W \cdot k & k_W \cdot k \end{bmatrix}$$

$$\Rightarrow P_C = t + R P_W$$

Perspective Projection

- Steps

- 3. Transform P_W into P_C

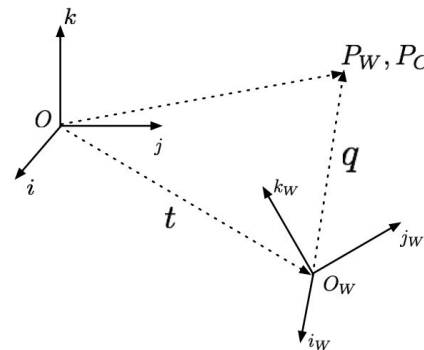
- In **homogeneous coordinates**

$$\Rightarrow P_C = t + R P_W$$

$$\begin{pmatrix} P_C \\ 1 \end{pmatrix} = \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix} \begin{pmatrix} P_W \\ 1 \end{pmatrix}$$

Point P_C in homogeneous coordinates

Point P_W in homogeneous coordinates



Perspective Projection

- Collecting all results

$$p^h = [K \quad 0_{3 \times 1}] P_C^h = K [I_{3 \times 3} \quad 0_{3 \times 1}] \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix} P_W^h$$

- Hence

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \overbrace{p^h = K [R \quad t] P_W^h}^{\text{Projection matrix } M} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Intrinsic parameters Extrinsic parameters

R: rotation
t: translation
Rt: extrinsic matrix

Degree of freedom:
K: 4
Rt: 6

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)

Blueprint attribute	Type	Default	Description
bloom_intensity	float	0.675	Intensity for the bloom post-process effect, 0.0 for disabled.
fov	float	90.0	Horizontal field of view in degrees.
fstop	float	1.4	Opening of the camera lens. Aperture is 1/fstop with typical lens going down to f/1.2 (larger opening). Larger values result in more depth of field.
image_size_x	int	800	Image width in pixels.
image_size_y	int	600	Image height in pixels.
iso	float	100.0	The camera sensor sensitivity.
gamma	float	2.2	Target gamma value of the camera.
lens_flare_intensity	float	0.1	Intensity for the lens flare post-process effect, 0.0 for disabled.
sensor_tick	float	0.0	Simulation seconds between sensor captures (ticks).
shutter_speed	float	200.0	The camera shutter speed in seconds (1.0/s).

Projection matrix M

$$p^h = K[R \quad t]P_W^h$$

Intrinsic parameters Extrinsic parameters

R: rotation
t: translation
Rt: extrinsic matrix

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?

Blueprint attribute	Type	Default	Description
bloom_intensity	float	0.675	Intensity for the bloom post-process effect, 0.0 for disabled.
fov	float	90.0	Horizontal field of view in degrees.
fstop	float	1.4	Opening of the camera lens. Aperture is 1/fstop with typical lens going down to f/1.2 (larger opening). Larger fstop values result in a shallower depth of field.
image_size_x	int	800	Image width in pixels.
image_size_y	int	600	Image height in pixels.
iso	float	100.0	The camera sensor sensitivity.
gamma	float	2.2	Target gamma value of the camera.
lens_flare_intensity	float	0.1	Intensity for the lens flare post-process effect, 0.0 for disabled.
sensor_tick	float	0.0	Simulation seconds between sensor captures (ticks).
shutter_speed	float	200.0	The camera shutter speed in seconds (1.0/s).

Projection matrix M

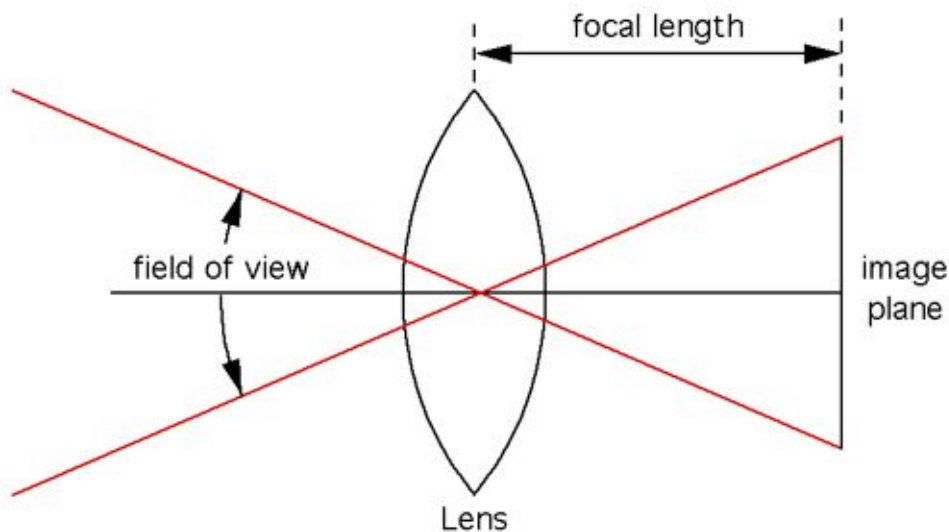
$$p^h = K[R \quad t]P_W^h$$

Intrinsic parameters Extrinsic parameters

R: rotation
t: translation
Rt: extrinsic matrix

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?
 - We can compute that



horizontal field of view = $2 \operatorname{atan}(0.5 \text{ width} / \text{focallength})$

Python code:

```
Focus_length = ImageSizeX / (2 * tan(CameraFOV * π / 360))  
Center_X = ImageSizeX / 2  
Center_Y = ImageSizeY / 2
```

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?
 - Here is the full intrinsic in Carla

Python code:

```
Focus_length = ImageSizeX / (2 * tan(CameraFOV * π / 360))  
Center_X = ImageSizeX / 2  
Center_Y = ImageSizeY / 2
```

```
K = [[f, 0, Cu],  
     [0, f, Cv],  
     [0, 0, 1 ]]
```

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?
 - Here is the full intrinsic in Carla

Python code:

```
Focus_length = ImageSizeX / (2 * tan(CameraFOV * π / 360))  
Center_X = ImageSizeX / 2  
Center_Y = ImageSizeY / 2
```

```
K = [[f, 0, Cu],  
     [0, f, Cv],  
     [0, 0, 1 ]]
```

How to do inverse perspective projection? Given $p_{2d} = [u, v, 1]$, how to get world point $P = [X, Y, Z]$?

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?
 - Here is the full intrinsic in Carla

Python code:

```
K = [[f, 0, Cu],  
      [0, f, Cv],  
      [0, 0, 1 ]]
```

```
def compute_intrinsic(img_width, img_height, fov):  
    """Compute intrinsic matrix."""  
    intrinsic = np.identity(3)  
    intrinsic[0, 2] = img_width / 2.0  
    intrinsic[1, 2] = img_height / 2.0  
    intrinsic[0, 0] = intrinsic[1, 1] = img_width / (2.0 * np.tan(fov *  
                                                                    np.pi / 360.0))  
  
    return intrinsic
```

How to do inverse perspective projection? Given $p2d = [u, v, 1]$, how to get world point $P = [X, Y, Z]$?
 R_t (camera extrinsic) is known in 3D simulation at all times

$$P = (\text{inv}(K) * p2d) * \text{depth} * R_t$$

Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Geometric computer vision
 - Camera models
 - Perspective projection
 - Camera calibration
 - Bird-Eye-View perception
 - Ego-View perception

Camera Calibration

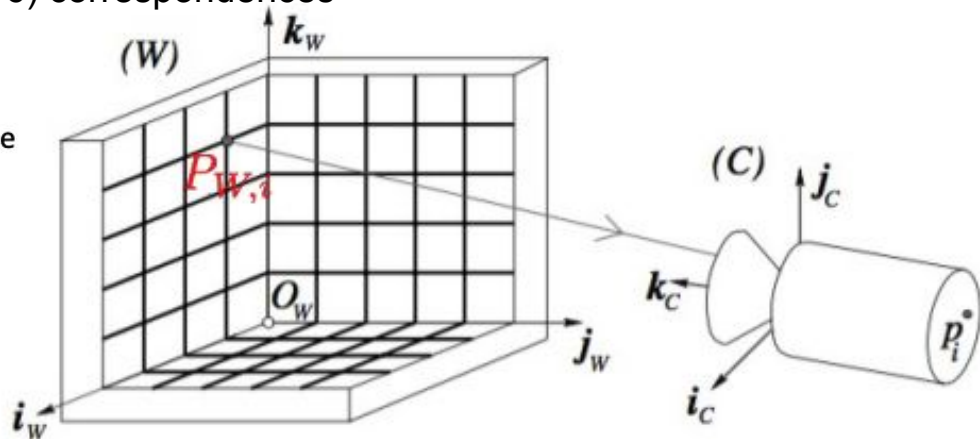
- The target for camera calibration is to get the **intrinsic and extrinsic parameters** of the camera

- We need to know 10* parameters (4 for K, 3 for R, 3 for t)
- Strategy
 - We need enough (at least 6) correspondences

$$p_i \leftrightarrow P_{W,i}$$

$P_{W,1}, P_{W,2}, \dots, P_{W,n}$ with **known** positions in world frame

$p_1, p_2, \dots, p_n$ with **known** positions in image frame



* or 11 if we include skewness

Camera Calibration

- The target for camera calibration is to get the **intrinsic and extrinsic parameters** of the camera

- We need to know 10* parameters (4 for K, 3 for R, 3 for t)

- Strategy

- We need enough (at least 6) correspondences

$$p_i \leftrightarrow P_{W,i}$$

- Method

- With the help of a chessboard and OpenCV...

- It is easy to compute correspondences for some patterns

- https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html



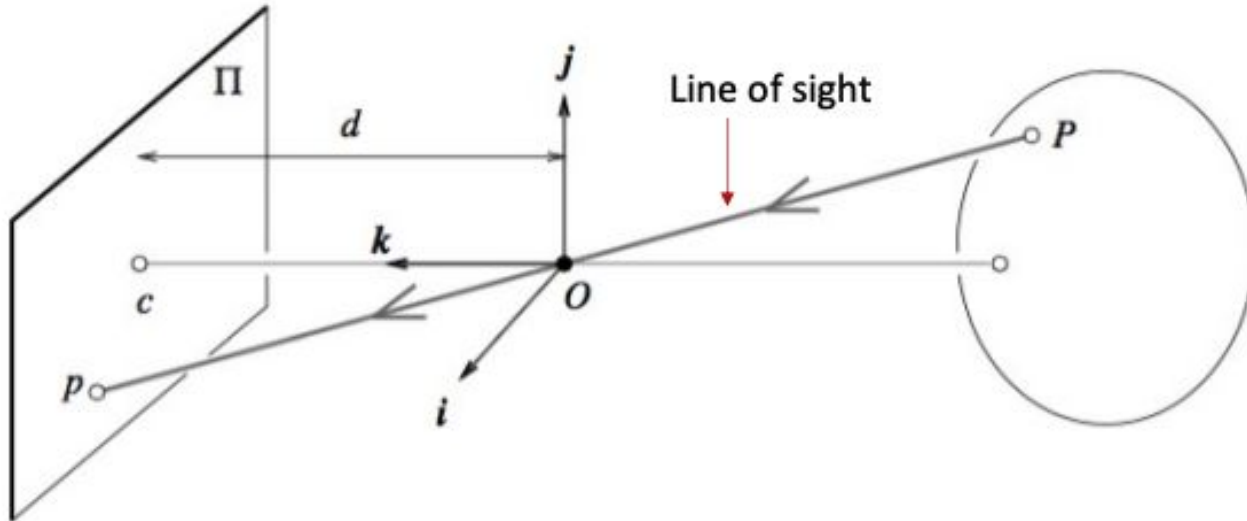
`cv.findChessboardCorners()`

`cv.calibrateCamera()`

* or 11 if we include skewness

Measuring depth - 3D reconstruction

- If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?
 - No! We can only know P is on the line OP



Measuring depth - 3D reconstruction

- If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?
 - No! We can only know P is on the line OP
 - So how?
 - **Stereo vision**
 - Two cameras, relative pose is known (like our eyes)
 - **Structure from motion**
 - Same camera, multiple images
 - **LiDAR!**

AI Applications - From Our UG Summer Research Project

Since now we can detect an object (like a tennis ball). Can we estimate its speed?

Answer: Yes! With a depth camera!



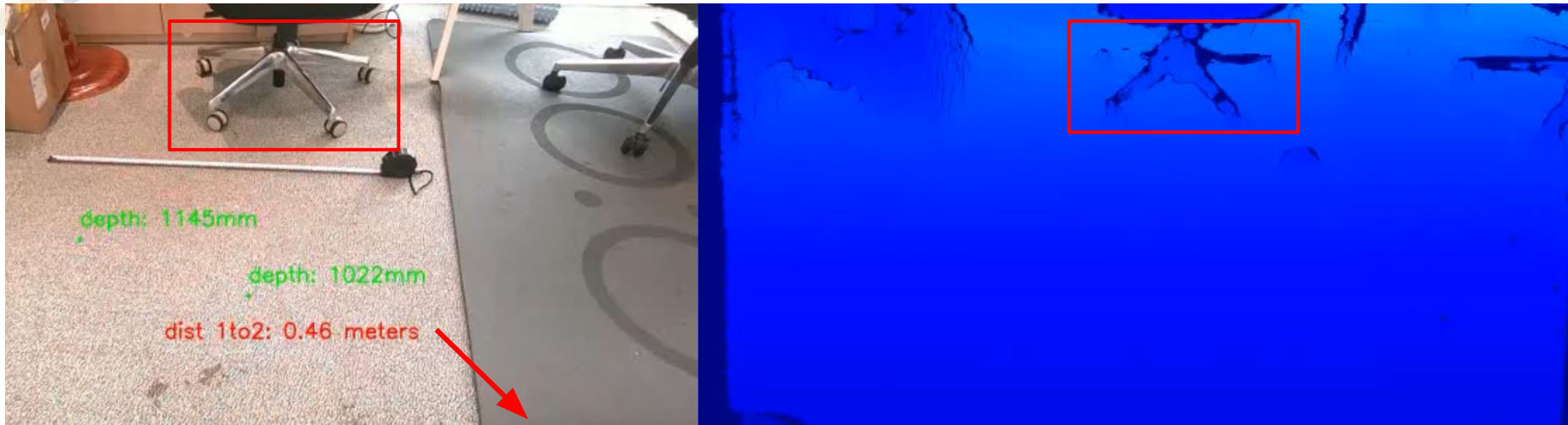
Speed?



AI Applications - From Our UG Summer Research Project

Since now we can detect an object (like a tennis ball). Can we estimate its speed?

Answer: Yes! With a depth camera!



The depth camera tells you how far away each pixel is. So you can compute real-world distance measurements.

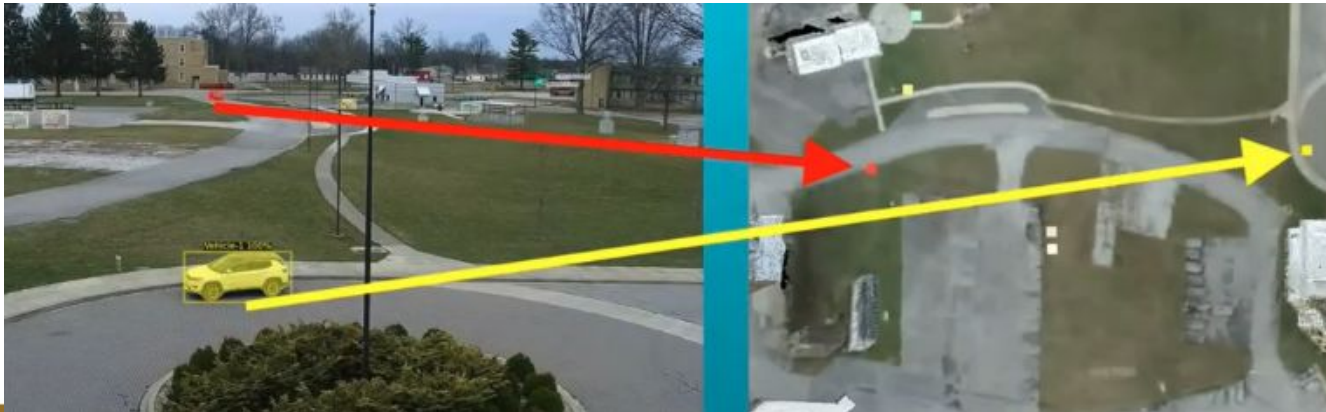
Measuring depth - 3D reconstruction

- If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?
 - No! We can only know P is on the line OP
 - But we can still get from **image plane to the ground plane** (BEV!!)
 - Need to know the ground plane's parameters
 - Homography matrix = $K_2 * R_{21} * \text{inv}(K_1)$
 - This allow you to..

Need to know: Intrinsics of the image camera and the ground plane camera + the relative extrinsics

Measuring depth - 3D reconstruction

- If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?
 - No! We can only know P is on the line OP
 - But we can still get from **image plane to the ground plane** (Bird Eye View!!)
 - Need to know the ground plane's parameters
 - Homography matrix = $K_2 * R_{21} * \text{inv}(K_1)$
 - This allow you to..

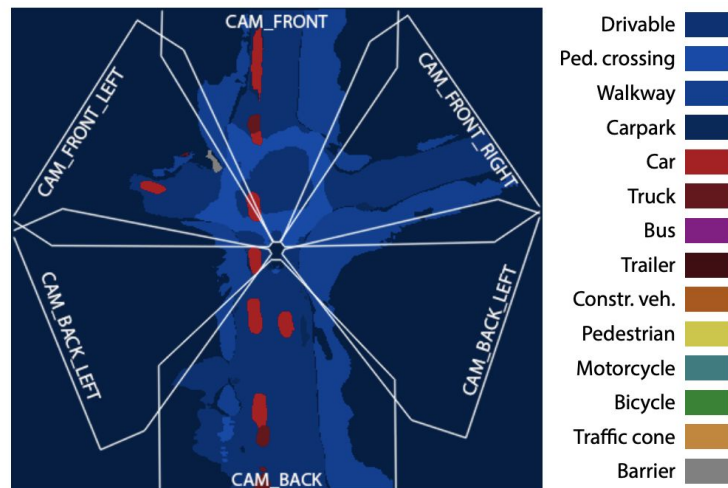


Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Geometric computer vision
 - Camera models
 - Perspective projection
 - Camera calibration
 - Bird-Eye-View perception
 - Ego-View perception

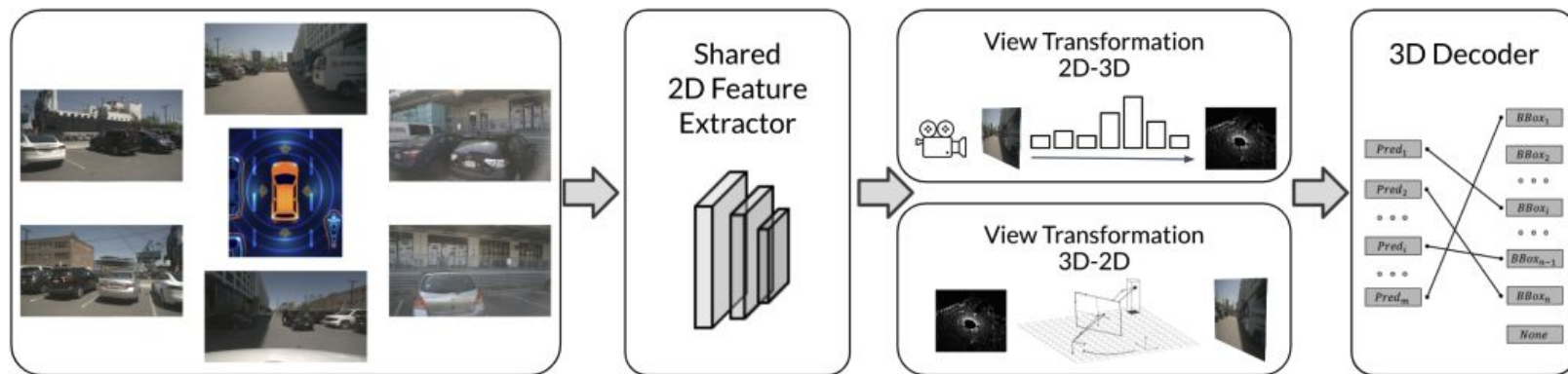
Bird-Eye-View Perception

- What is BEV perception?
 - Previously, detection/segmentation/tracking, etc., done in a front view or perspective view
 - With more cameras and sensors, need a unified view
 - From multi-camera/sensor to a top-down unified view (BEV)
 - Increasingly popular for autonomous driving algorithms



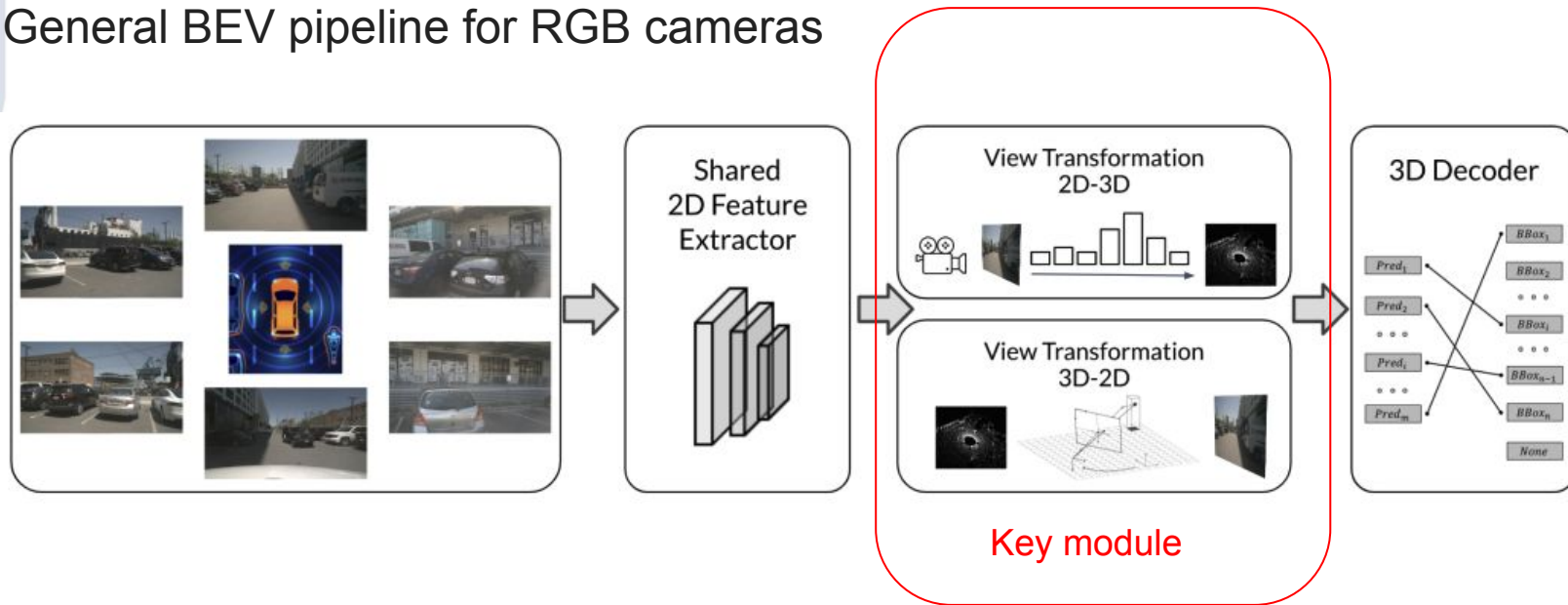
Bird-Eye-View Perception

- General BEV pipeline for RGB cameras



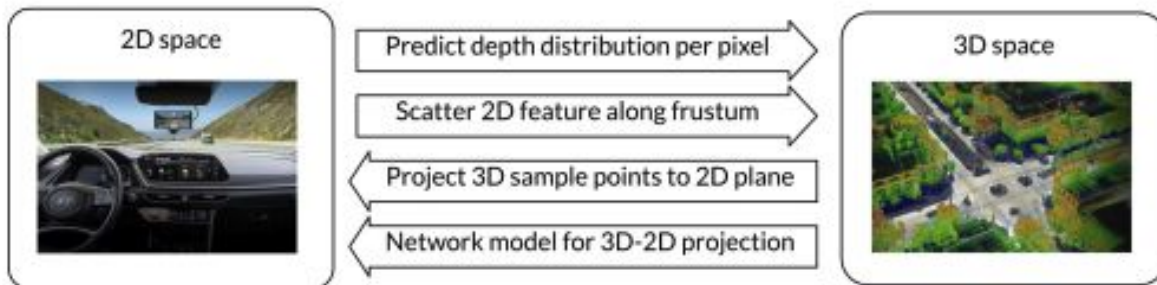
Bird-Eye-View Perception

- General BEV pipeline for RGB cameras



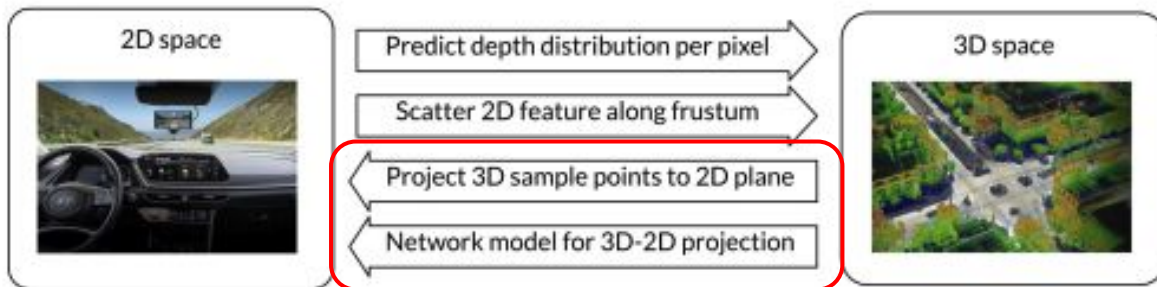
Bird-Eye-View Perception

- General BEV pipeline for RGB cameras
 - 2D-3D or 3D-2D?



Bird-Eye-View Perception

- General BEV pipeline for RGB cameras
 - 2D-3D or 3D-2D?

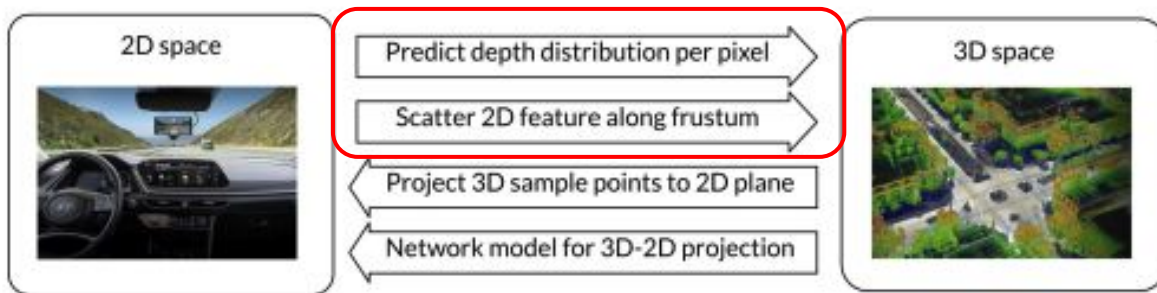


From 3D to 2D

1. On the BEV map, for each (3D) point, project to the camera 2D plane using camera parameters
2. Sample feature from 2D space

Bird-Eye-View Perception

- General BEV pipeline for RGB cameras
 - 2D-3D or 3D-2D?

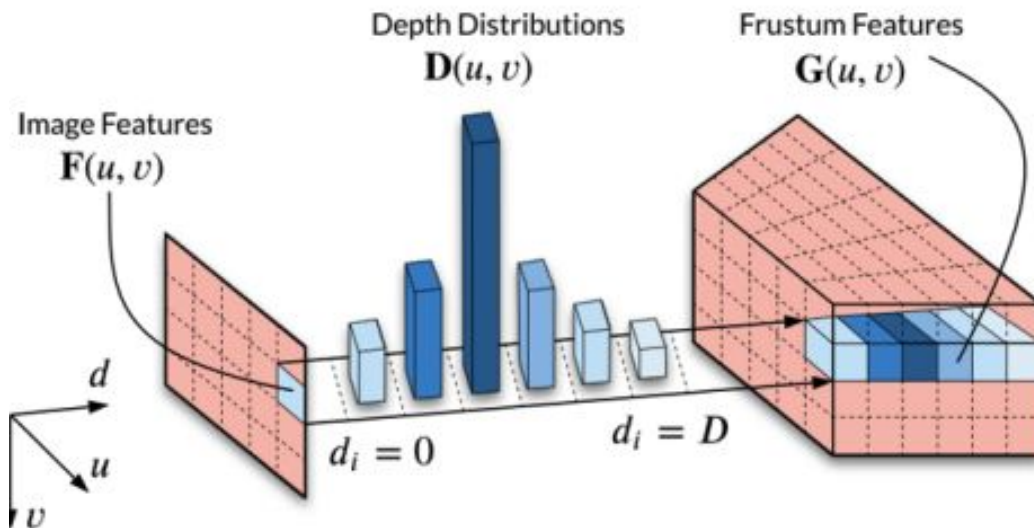


From 2D to 3D?

How do we get feature in 3D space directly?

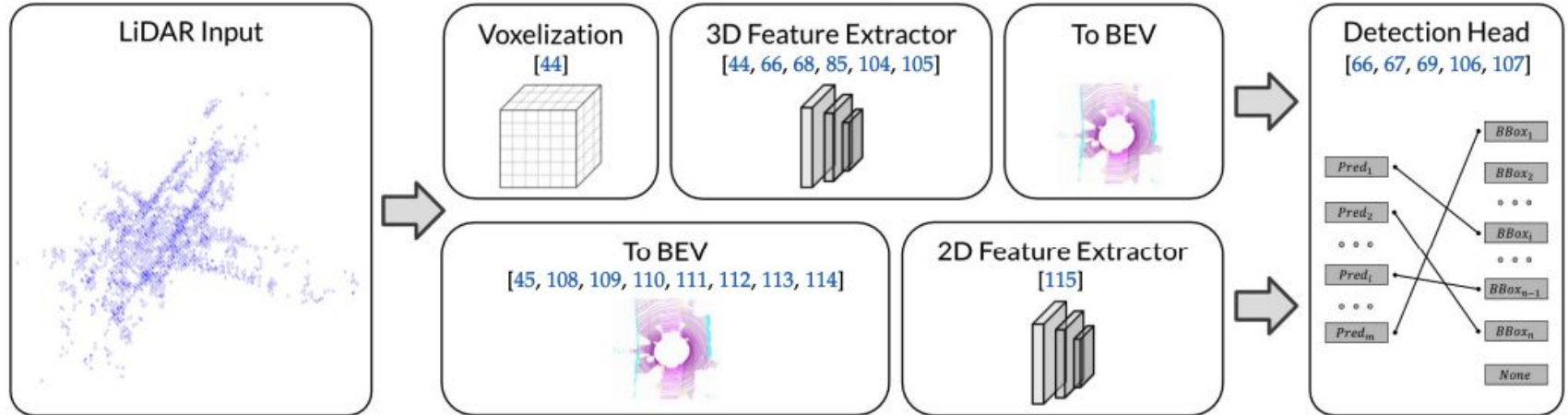
Bird-Eye-View Perception

- General BEV pipeline for RGB cameras
 - 2D-3D
 - Predict per-pixel depth as feature distribution along the projection cone
 - 2D features -> 3D voxels



Bird-Eye-View Perception

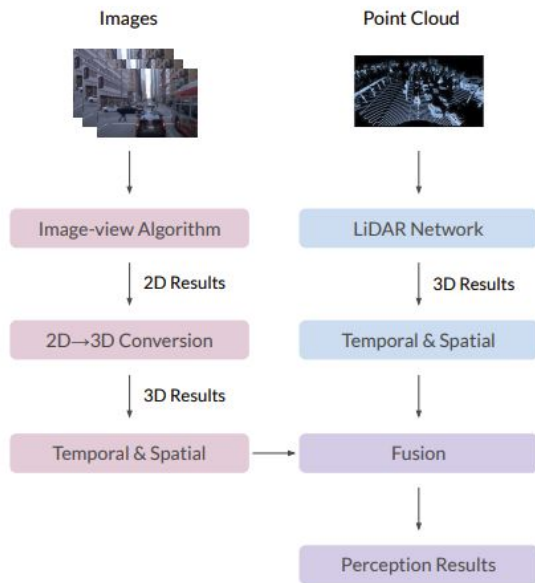
- General BEV pipeline for LIDAR / Radar
 - LIDAR inputs are point clouds in 3D



Bird-Eye-View Perception

- Fusing RGB and LIDAR
 - Before

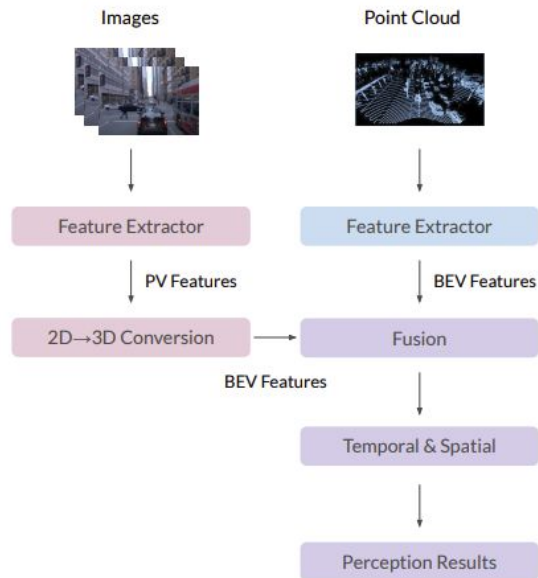
Late fusion



(a) Perspective view (PV) perception pipeline

Bird-Eye-View Perception

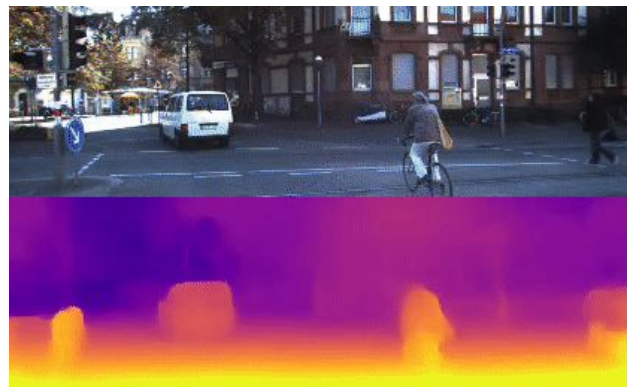
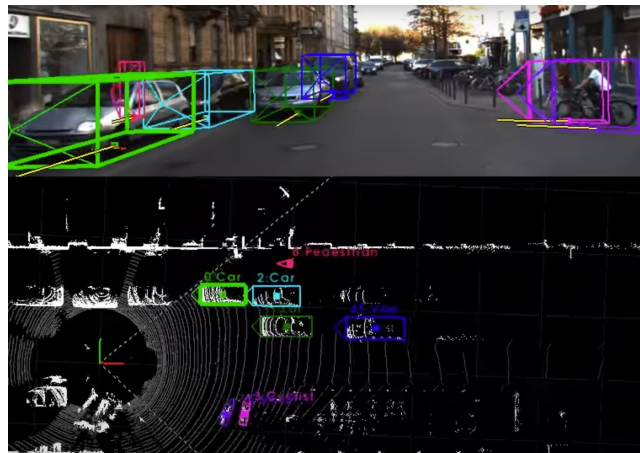
- Fusing RGB and LIDAR
 - Now



(b) BEV perception pipeline (BEV Fusion)

Bird-Eye-View Perception

- BEV Perception datasets
 - KITTI, - RGB frames, 3D bbox, Depth*



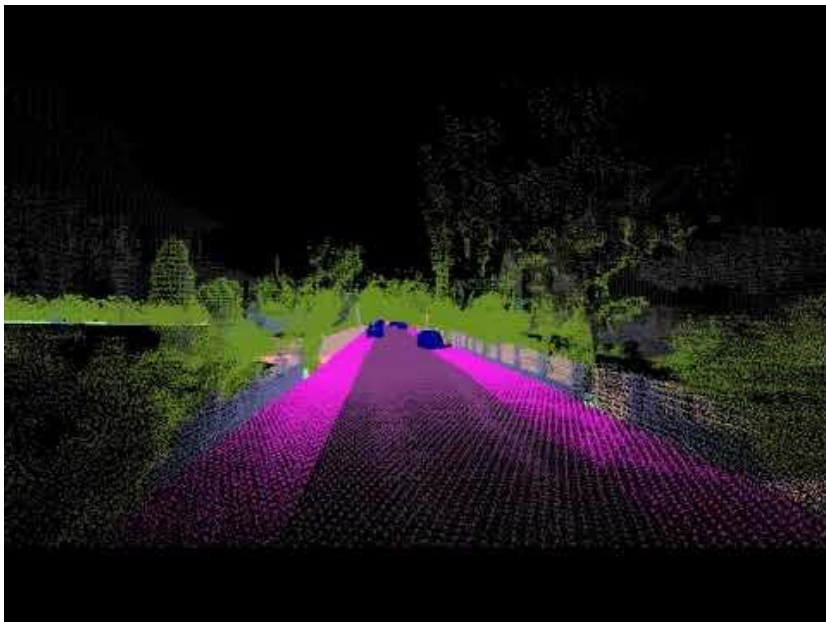
Bird-Eye-View Perception

- BEV Perception datasets
 - SemanticKITTI - RGB frames, 3D bbox, 3D segmentation



Bird-Eye-View Perception

- BEV Perception datasets
 - KITTI-360 - 2019



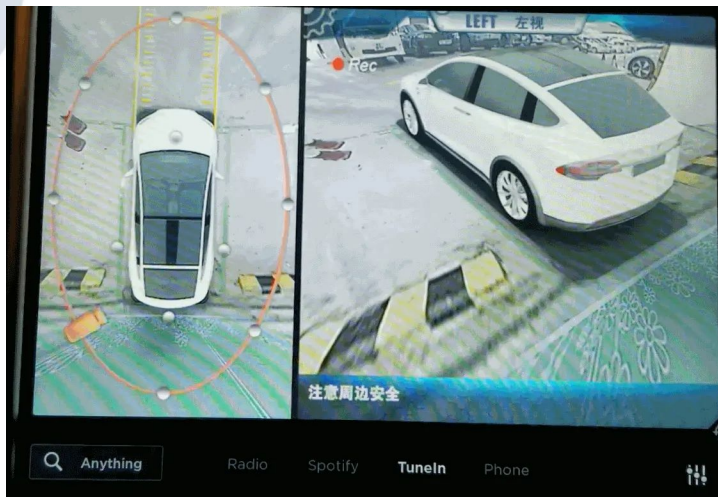
Bird-Eye-View Perception

- BEV Perception datasets
 - And more
 - nuScenes
 - Waymo Open Dataset
 - Argo

Dataset	Year	City
KITTI [11]	2012	EU
Waymo [8]	2019	NA
nuScenes [7]	2019	NA/AS
Argo v1 [24]	2019	NA
Argo v2 [12]	2022	NA
ApolloScape [25]	2018	AS
OpenLane [26]	2022	NA
ONCE-3DLanes [27]	2022	AS
Lyft L5 [28]	2019	AF
A* 3D [29]	2019	AS
H3D [30]	2019	NA
SemanticKITTI [31]	2019	EU
A2D2 [32]	2020	EU
Cityscapes 3D [33]	2020	-
PandaSet [34]	2020	NA
KITTI-360 [35]	2020	EU
Cirrus [36]	2020	-
ONCE [37]	2021	AS
AIODrive [38]	2021	Sim
DeepAccident [39]	2022	Sim

Bird-Eye-View Perception

- Existing BEV applications



Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Geometric computer vision
 - Camera models
 - Perspective projection
 - Camera calibration
 - Bird-Eye-View perception
 - Ego-View perception

Egocentric Perception

- Or perspective view perception / First-person view perception
 - For robot vision and augmented reality



First
Person



Different from...

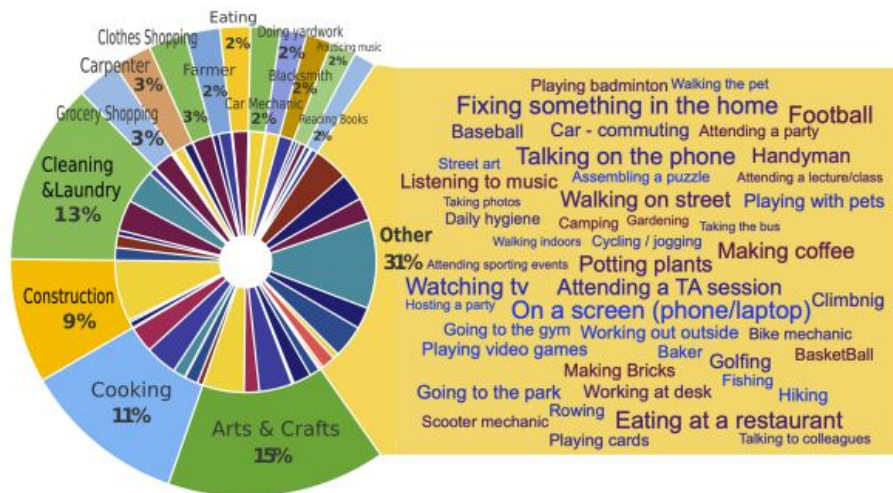
Third
Person



Egocentric Perception

- Ego4D dataset

- 3000+ hours of ego-centric videos (head-mounted go-pros, etc.) over 74 locations in the world
- Daily activities:



[1] Ego4D: Around the World in 3,000 Hours of Egocentric Video. CVPR 2022

[2] <https://ego4d-data.org/>

Egocentric Perception

- Ego4D dataset
 - 3000+ hours of ego-centric videos (head-mounted go-pros, etc.) over 74 locations in the world
 - Task: Hands and Objects interaction
 - For robot demonstration learning!



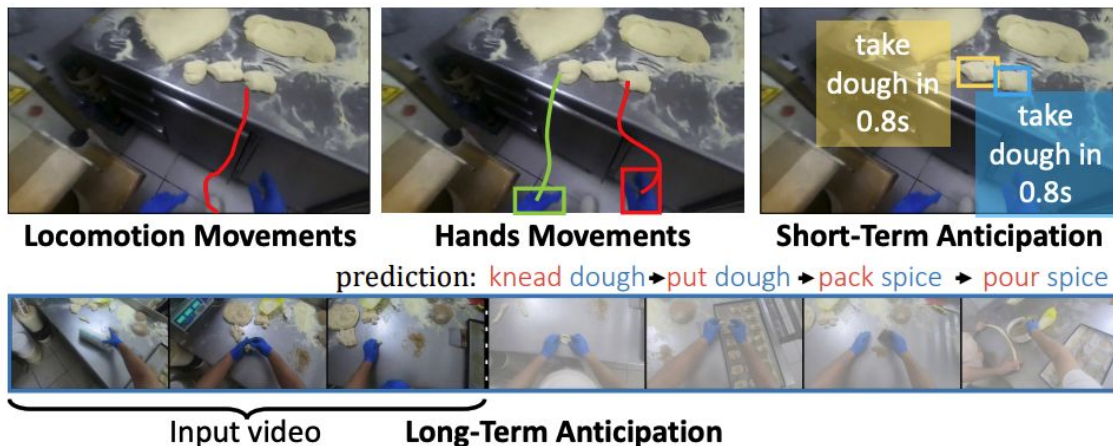
State-change: Plant removed from ground

[1] Ego4D: Around the World in 3,000 Hours of Egocentric Video. CVPR 2022

[2] <https://ego4d-data.org/>

Egocentric Perception

- Ego4D dataset
 - 3000+ hours of ego-centric videos (head-mounted go-pros, etc.) over 74 locations in the world
 - Task: Forecasting
 - Predict what the person will do, their hand movements - world model!

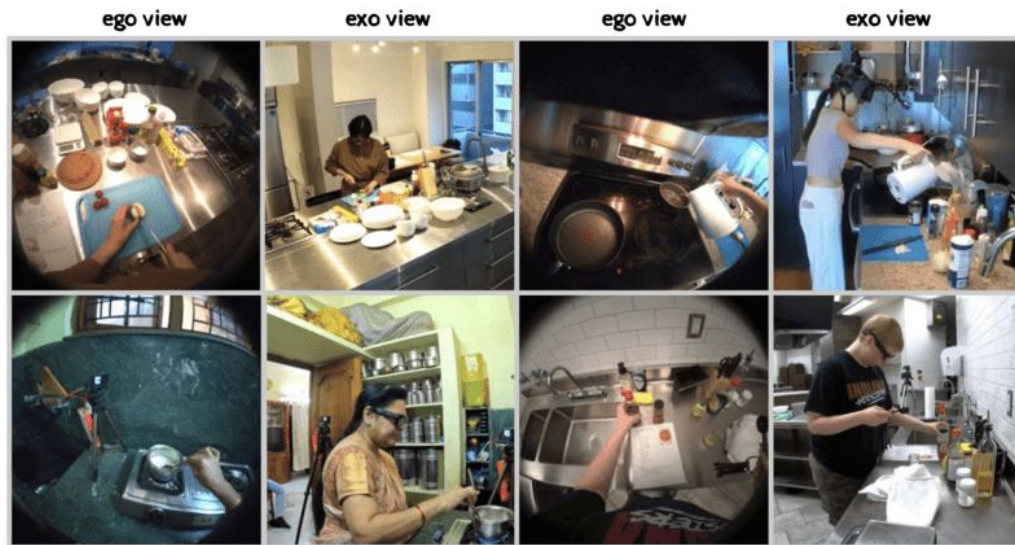


[1] Ego4D: Around the World in 3,000 Hours of Egocentric Video. CVPR 2022

[2] <https://ego4d-data.org/>

Egocentric Perception

- Ego-Exo4D dataset
 - First-person + third person video data



[1] Ego-Exo4D: Understanding Skilled Human Activity from First- and Third-Person Perspectives. Dec 2023

[2] <https://ego-exo4d-data.org/>

Egocentric Perception

- Ego-Exo4D dataset
 - First-person + third person video data
 - Train a pretrain model for how to accomplish certain tasks from human demonstrations

Bike Repair



Narrate and Act: It's nice and tight, clockwise to tighten that drive side nut, do another quick check.

Atomic Action Description: C tightens the axle nut with the bicycle skewer in his right hand.

Expert Commentary: Now the mechanic is applying pressure to the left side of the wheel and pushing the wheel towards the right. That allows to have the cog pull the chain and create a little more tension on the chain. Meanwhile, he is tightening the right side nut and making sure that there is some tension on the chain allowing a little slack but also holding it into place.

[1] Ego-Exo4D: Understanding Skilled Human Activity from First- and Third-Person Perspectives. Dec 2023

[2] <https://ego-exo4d-data.org/>

Agenda

- Objectives
 - We will learn the key perception models of Embodied AI
- The tasks
 - Computer vision background
 - Image and Video Storage
 - Pre-CNN features
 - Visual recognition
 - Geometric computer vision
 - Camera models
 - Perspective projection
 - Camera calibration
 - Bird-Eye-View perception
 - Ego-View perception

Reading Material

- BEV perception survey
 - Delving into the Devils of Bird's-eye-view Perception: A Review, Evaluation and Recipe. 2022
 - Vision-Centric BEV Perception: A Survey, 2022
- Camera calibration tutorial
 - https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html
 - https://boofcv.org/index.php?title=Tutorial_Camera_Calibration
 - https://github.com/zhengthomastang/Cal_PnP
- EPro-PnP: Generalized End-to-End Probabilistic Perspective-n-Points for Monocular Object Pose Estimation, CVPR 2022 best paper
 - Calibrate objects in the image



Thank you!